SRI VENKATESWARA GOVERNMENT POLYTECHNIC TIRUPATI – 517501, ANDHRA PRADESH DEPARTMENT OF COMPUTER ENGINEERING

ACADEXSYNC A Modern Multi-Tenant Campus Management Operating System

A Project Report Submitted to the STATE BOARD OF TECHNICAL EDUCATION AND TRAINING, ANDHRA PRADESH In partial fulfillment of the requirements for the award of DIPLOMA IN COMPUTER ENGINEERING (2024–2027)

Submitted by:

Table 1: Project Team Members

S. No.	Student Name	PIN Number
1	S. Venkata Kalyan	24018-CM-062
2	R. Poornesh	24018-CM-057
3	R. Naga Rasagna	24018-CM-006
4	L. Harsha Vardhan	24018-CM-046
5	K. Uday Kumar	24018-CM-045
6	N. Santosh	24018-CM-052

Under the Esteemed Guidance of: B. MOHAN, M.Tech Lecturer, Department of Computer Engineering (CME) Sri Venkateswara Government Polytechnic, Tirupati Academic Year 2026–2027

SRI VENKATESWARA GOVERNMENT POLYTECHNIC TIRUPATI – 517501, ANDHRA PRADESH DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the technical project work entitled "ACADEXSYNC: MULTI-TENANT CAMPUS MANAGEMENT APPLICATION" is a bonafide project work carried out by the following students of final-year Diploma in Computer Engineering:

Table 2: Certified Project Candidates

S. No.	Student Name	PIN Number
1	S. Venkata Kalyan	24018-CM-062
2	R. Poornesh	24018-CM-057
3	R. Naga Rasagna	24018-CM-006
4	L. Harsha Vardhan	24018-CM-046
5	K. Uday Kumar	24018-CM-045

6	N. Santosh	24018-CM-052

This project is submitted in partial fulfillment of the requirements for the award of DIPLOMA IN COMPUTER ENGINEERING conferred by the State Board of Technical Education and Training (SBTET), Andhra Pradesh, during the academic period 2024–2027. The project work has been scrutinized and approved under our supervision.

Project Guide	Head of Department	Principal
_____	_____	_____
B. MOHAN, M.Tech Lecturer in CME Dept. of Computer Engg.	G. PRABAKAR, M.Tech Head of Section (HCME) Dept. of Computer Engg.	Dr. SRINIVASULU, M.Tech, Ph.D Principal SV Govt. Polytechnic, Tirupati

Date: _____ Place: Tirupati, Andhra Pradesh

DECLARATION

We hereby declare that the project report entitled "ACADEXSYNC: MULTI-TENANT CAMPUS MANAGEMENT APPLICATION" has been prepared by our project team under the guidance of our project guide B. Mohan, M.Tech, Lecturer in the Department of Computer Engineering, Sri Venkateswara Government Polytechnic, Tirupati. It is submitted in partial fulfillment of the academic curriculum for the award of Diploma in Computer Engineering under the State Board of Technical Education and Training (SBTET), Andhra Pradesh, for the academic year 2026–2027.

We affirm that this project report is an authentic record of our own engineering design, architectural synthesis, and verified software development. The technical specifications, system architecture models, database schemas, API contracts, and source-code listings presented herein are directly derived from the actual ACADEXSYNC codebase, comprising our cross-platform Flutter application, Node.js/TypeScript Express backend, and Python FastAPI microservices. This work has not been submitted previously to any other university or examining board for the award of any diploma or degree.

Candidate Signatures:

1. S. Venkata Kalyan (24018-CM-062) Signature: _____	2. R. Poornesh (24018-CM-057) Signature: _____
3. R. Naga Rasagna (24018-CM-006) Signature: _____	4. L. Harsha Vardhan (24018-CM-046) Signature: _____
5. K. Uday Kumar (24018-CM-045) Signature: _____	6. N. Santosh (24018-CM-052) Signature: _____

ACKNOWLEDGEMENT

The successful completion of this major final-year project report would not have been possible without the invaluable guidance, constructive criticism, and continuous encouragement provided by numerous individuals and institutional mentors.

First and foremost, we express our deep sense of gratitude and sincere thanks to our esteemed Project Guide, B. Mohan, M.Tech, Lecturer in the Department of Computer Engineering, for his exemplary guidance, patient technical advice, architectural insights, and inspiring suggestions throughout the development of ACADEXSYNC. His rigorous emphasis on software engineering discipline and software reliability helped us surmount critical engineering hurdles.

We wish to express our heartfelt gratefulness and profound thanks to G. Prabakar, M.Tech, Head of Computer Engineering Section (HCME), for his invaluable administrative support, technical leadership, and academic coordination during the course of our diploma study.

We have great pleasure in expressing our sincere and respectful gratitude to our beloved Principal, Dr. Srinivasulu, M.Tech, Ph.D., for providing excellent institutional facilities, well-equipped computer laboratories, continuous inspiration, and the academic environment required to engineer an enterprise-grade campus management operating system.

We also extend our earnest thanks to all the faculty members, non-teaching staff, and laboratory technicians of the Department of Computer Engineering for their direct and indirect cooperation. Finally, we express our eternal love and heartfelt gratitude to our parents and friends whose unwavering moral and emotional support enabled us to successfully realize this project.

ABSTRACT

ACADEXSYNC is an enterprise-grade, multi-tenant Campus Management Operating System engineered to resolve the severe operational fragmentation, data desynchronization, and administrative vulnerabilities inherent in conventional collegiate workflows. Traditional educational institutions rely heavily upon disjointed paper registers, standalone spreadsheets, isolated departmental notices, and fragmented chat groups. This fragmented model frequently results in attendance tampering, double-booked teaching allocations, prolonged student grievance cycles, and an absence of unified institutional oversight.

To decisively overcome these challenges, ACADEXSYNC introduces a secure, multi-tier distributed architecture spanning three integrated subsystems: (1) A high-performance, cross-platform client developed with Google Flutter (3.x) and Dart, featuring Riverpod state management, GoRouter declarative route guards, and mobile-first responsive layouts; (2) An enterprise RESTful API gateway built on Node.js, Express.js, and TypeScript, enforcing strict tenant scoping, JSON Web Token (JWT) cryptographic validation, and

authoritative stateful session tracking via an immutable AuthSession store; and (3) An AI microservice layer leveraging Google Gemini 1.5 Flash SDK function-calling and Python FastAPI for timetable constraint solving and contextual retrieval.

The system enforces a rigorous five-tier Role-Based Access Control (RBAC) hierarchy comprising Super Admin (platform-level governance), College Admin (institutional-level tenant management), Head of Department (departmental academic and workload coordination), Faculty (period-wise roll call, session locking, and syllabus management), and Student (personal academic tracking, digital attendance audit, and request submission). Data isolation is guaranteed across all 37 canonical domain models in MongoDB Atlas through mandatory tenant scoping (`requireCollegeScope`) and compound database indices ({ collegeId: 1, sectionId: 1, date: 1 }).

Key operational innovations include: (a) An idempotent attendance ingestion engine with cryptographic Unicode string normalization, preventing duplicate timeslot entries and enforcing strict completeness invariants (present + late + absent + excused + unmarked == total); (b) A deterministic timetable validator that detects faculty double-booking, room capacity clashes, and break overlaps in real-time; (c) A formalized Request Center with multi-level role routing and immutable audit tracking; and (d) Multi-channel real-time notification dispatch integrating Firebase Cloud Messaging (FCM) multicast pushes and Server-Sent Events (SSE). The system has undergone exhaustive unit, integration, and widget testing across 131 GoRouter paths and 51 API endpoints, confirming zero cross-tenant leakage and production-ready reliability.

TABLE OF CONTENTS

Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

List of Figures viii

List of Tables x

List of Abbreviations xii

CHAPTER 1: INTRODUCTION 1

1.1 Digital Transformation in Educational Administration 1

1.2 Problems with Manual Registers and Disconnected Records 3

1.3 Purpose and Vision of ACADEXSYNC 5

1.4 Need for Centralized Academic Information 7

1.5 Target Users and Institutional Context 9

1.6 Scope and Project Limitations 11

CHAPTER 2: PROJECT DESCRIPTION 13

2.1 Problem Statement 13

2.2 Project Objectives 15

2.3 Project Scope 17

2.4 Motivation for Selecting the Project 18

2.5 Proposed Solution Overview 20

2.6 Expected Quantitative & Qualitative Benefits 22

2.7 Functional Requirements 24

2.8 Non-Functional Requirements 27

2.9 Assumptions and Technical Dependencies 30

CHAPTER 3: COMPUTATIONAL ENVIRONMENT 32

3.1 Hardware Requirements (Server & Client Devices) 32

3.2 Software Requirements & Version Specifications 34

3.3 Technology Stack Evaluation & Architectural Rationale 37

CHAPTER 4: FEASIBILITY STUDY 40

4.1 Technical Feasibility 40

4.2 Operational Feasibility 42

4.3 Economic Feasibility 44

4.4 Maintenance Feasibility 46

4.5 Deployment and Scaling Considerations 48

CHAPTER 5: SYSTEM ANALYSIS 50

5.1 Existing Academic Management Approaches 50

5.2 Deficiencies and Risks of Conventional Systems 52

5.3 Proposed ACADEXSYNC Solution 55

5.4 Distinctive Advantages of Proposed Architecture 57

5.5 Comprehensive Comparative Evaluation Matrix 59

5.6 Core Operational Workflows 62

CHAPTER 6: SYSTEM DESIGN AND ARCHITECTURE 65

6.1 Overall System Architecture 65

6.2 Frontend Architecture (Flutter / Riverpod) 68

6.3 Backend Architecture (Express.js / TypeScript) 71

6.4 Database Architecture & Entity Modeling 74

6.5 Authentication and Stateful Session Store 77

6.6 Multi-Tenant Isolation & Workspace Architecture 80

6.7 Role and Permission Architecture 83

6.8 API Communication Flow & Contracts 86

6.9 Attendance Processing and Session-Locking Flow 89

6.10 Timetable Management & Conflict Resolution Flow 92

6.11 Request Center Multi-Tier Workflow 95

6.12 Notification Delivery & FCM Multicast Flow 98

6.13 AI Assistant Request Flow & Tool Execution 101

6.14 UML Diagrams & Architectural Blueprints 104

CHAPTER 7: ROLE-BASED ACCESS & INSTITUTIONAL GOVERNANCE 108

7.1 Super Admin (Platform-Level Governance) 108

7.2 College Admin (Institutional Governance) 111

7.3 Head of Department (Departmental Academic Oversight) 114

7.4 Faculty (Academic Operations & Roll Call) 117

7.5 Student (Personal Academic Access & Requests) 120

7.6 Role and Permission Authorization Matrix 123

7.7 Role Hierarchy & Security Boundaries 126

7.8 Multi-Tenant Isolation & Workspace Switching 129

CHAPTER 8: SYSTEM IMPLEMENTATION 132

8.1 Application Initialization & Bootstrapping 132

8.2 Authentication & Account Activation Module 135

8.3 Dynamic Role-Based Navigation Routing 138

8.4 Super Admin Management Workstation 141

8.5 College Admin Institutional Console 144

8.6 Head of Department Academic Portal 147

8.7 Faculty Classroom Operations Suite 150

8.8 Student Academic Hub & Attendance Portal 153

8.9 Attendance Ingestion & Idempotent Session Locking 156

8.10 Timetable, Substitution & Calendar Override Engine 159

8.11 Academic Structure & Curriculum Configuration 162

8.12 Notes Library & ImageKit Secure Storage 165

8.13 Institutional Announcements & FCM Multicast 168

8.14 Request Center & Multi-Level Review Workflow 171

8.15 AI Assistant & RAG Tool Calling Registry 174

8.16 Error Handling, Structured Logging & Validation 177

CHAPTER 9: IMPORTANT SOURCE CODE WITH EXPLANATION 180

Listing 1: Application Entry Point (`main.dart`) 180

Listing 2: Declarative Router & Route Guards (`app_router.dart`) 184

Listing 3: Role Definitions & Hierarchy (`roles.ts`) 188

Listing 4: Authentication & Session Middleware (`auth.middleware.ts`) 191

Listing 5: Super Admin & College Admin Selection (`app_router.dart`) 195

Listing 6: HOD, Faculty & Student Selection (`app_router.dart`) 198

Listing 7: Authentication Controller (`auth.controller.ts`) 201

Listing 8: Role Authorization Guard (`role.middleware.ts`) 204

Listing 9: Multi-Tenant College Scope Guard (`tenant.middleware.ts`) 207

Listing 10: Attendance Ingestion Pipeline (`attendance.service.ts`) 210

Listing 11: Idempotent Session Locking (`attendance.service.ts`) 213

Listing 12: Timetable Conflict Validator (`timetable.service.ts`) 216

Listing 13: Request Submission & Approval (`request.controller.ts`) 219

Listing 14: FCM Push Multicast Dispatch (`fcm.service.ts`) 222

Listing 15: Canonical Data Schema (`attendanceSession.model.ts`) 225

Listing 16: AI Gemini Tool Registry (`ai.service.ts`) 228

CHAPTER 10: UML DIAGRAMS, FLOWCHARTS & WORKFLOW MODELS 232

CHAPTER 11: TESTING AND VALIDATION 245

CHAPTER 12: RESULTS AND SCREEN LAYOUTS 260

CHAPTER 13: LIMITATIONS AND FUTURE ENHANCEMENTS 275

CHAPTER 14: CONCLUSION 280

REFERENCES & OFFICIAL DOCUMENTATION 285

APPENDIX A: BACKEND REST API ROUTE INVENTORY 288

APPENDIX B: DATABASE SCHEMA & FIELD DICTIONARY 292

APPENDIX C: TECHNICAL ABBREVIATIONS GLOSSARY 296

LIST OF FIGURES

Figure 6.1: ACADEXSYNC Overall Multi-Tenant System Architecture 67

Figure 6.2: ACADEXSYNC Subsystem Component Architecture 73

Figure 6.5: Authentication & Authoritative Session Sequence Diagram 79

Figure 6.6: User Account Activation & Credential Provisioning Sequence 82

Figure 6.7: Attendance Submission & Idempotent Session Locking Sequence 91

Figure 6.8: Timetable Construction & Conflict Validation Activity Diagram 94

Figure 6.9: Request Center Submission & Multi-Tier Approval Workflow 97

Figure 6.10: Multi-Channel Institutional Notification Dispatch Pipeline 100

Figure 6.11: Gemini AI Assistant Tool-Calling & Context Retrieval Flow 103

Figure 6.12: ACADEXSYNC Canonical Multi-Tenant Entity-Relationship Model 106

Figure 7.1: Multi-Tenant Identity & Request Authorization Pipeline 125

Figure 10.2: UML Use Case Diagram (ACADEXSYNC 5 User Roles) 234

Figure 12.1: ACADEXSYNC Mobile Application Launch Screen 261

Figure 12.2: Unified Authentication and Secure Sign-in Portal 262

Figure 12.3: Super Admin Platform Oversight Dashboard 263

Figure 12.4: Super Admin Multi-College Tenant Directory 264

Figure 12.5: Super Admin System-Wide Audit Log Monitor 265

Figure 12.6: Super Admin Gemini AI Assistant Interface 266

Figure 12.7: College Admin Institutional Operations Dashboard 267

Figure 12.8: College Admin Academic Hierarchy Setup Console 268

Figure 12.9: College Admin Institutional Staff & Student Directory 269

Figure 12.10: College Admin Comprehensive Analytics & Report Center 270

Figure 12.11: Head of Department (HOD) Academic Management Dashboard 271

Figure 12.12: Faculty Classroom Management & Teaching Workstation 272

Figure 12.13: Student Unified Academic Hub & Portal Dashboard 273

Figure 12.14: Student Dynamic Weekly Timetable & Period Schedule View 274

Figure 12.15: Student Subject-Wise Attendance Tracking & Breakdown Screen 274

LIST OF TABLES

Table 1: Project Team Member Information ..... Title Page

Table 2: Certified Project Candidates ..... Certificate

Table 3: Minimum & Recommended Hardware Specifications 33

Table 4: Verified Core Technology Software Stack & Versions 35

Table 5: Quantitative Comparison: Manual vs ACADEXSYNC Approaches 60

Table 6: Comprehensive 5-Tier Role & Permission Authorization Matrix 124

Table 7: Architectural Responsibility & Data Invariant Mapping 130

Table 8: Unit & Presentation Test Case Execution Matrix 248

Table 9: Backend API Integration & Role Security Test Matrix 252

Table 10: Attendance Invariant & Idempotent Session Lock Test Matrix 256

Table 11: Timetable Conflict & Schedule Overlap Test Matrix 258

LIST OF ABBREVIATIONS

Table 3: Technical Abbreviations and Expansions

Abbreviation	Complete Expansion
API	Application Programming Interface
CME	Computer Engineering Department
DAO	Data Access Object
DTO	Data Transfer Object
FCM	Firebase Cloud Messaging
HOD	Head of the Department
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model-View-Controller Architectural Pattern
ODM	Object Data Modeling
OTP	One-Time Password
PIN	Permanent Institutional Identification Number
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control

REST	Representational State Transfer
SBTET	State Board of Technical Education and Training
SDK	Software Development Kit
SSE	Server-Sent Events
UI	User Interface
UML	Unified Modeling Language
URI	Uniform Resource Identifier
UUID	Universally Unique Identifier
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Digital Transformation in Educational Administration

Over the past decade, the rapid evolution of information and communication technologies has revolutionized organizational operations across government, enterprise, and educational sectors. In higher education and polytechnic engineering institutions, digital transformation is no longer a peripheral luxury; it has become an indispensable imperative for academic integrity, institutional governance, and operational efficiency. Academic administration encompasses a vast web of highly interdependent responsibilities: scheduling lecture timetables across multiple physical classrooms and laboratories, recording daily and period-wise attendance, tracking continuous assessment marks, managing faculty workloads, coordinating curricular notes, and processing student applications for leave or duty exemptions. In traditional polytechnic institutions operating under state technical education boards, these processes have historically been administered through manual paper registers, static ledger books, and fragmented desktop spreadsheets.

While manual methods sufficed during earlier eras characterized by smaller student cohorts and fixed curricular regimes, the expansion of contemporary technical education has strained manual administration to its breaking point. Modern polytechnic colleges manage hundreds to thousands of students enrolled across diverse disciplines such as Computer Engineering, Mechanical Engineering, Civil Engineering, and Electronics. Under this operational scale, the continued reliance upon physical registers introduces intolerable friction, administrative overhead, human error, and vulnerability to record falsification. The digital transformation of campus management substitutes error-prone physical handling with automated, deterministic, and auditable digital workflows that operate synchronously across all institutional tiers.

1.2 Problems with Manual Attendance Registers and Disconnected Records

The core administrative vulnerability in collegiate operations lies in the capture and consolidation of student attendance. Under the manual paradigm, faculty members physically carry paper attendance ledgers from classroom to classroom during each period. This practice suffers from severe structural deficiencies:

Vulnerability to Proxies and Data Tampering: Physical attendance callouts in crowded classrooms are notoriously susceptible to fraudulent proxy responses. Furthermore, physical ledgers can be easily modified after the fact through unauthorized mark alterations, erasure, or unverified manual additions.

Severe Latency in Data Consolidation: Because attendance marks remain locked inside physical registers or individual faculty notebooks, aggregate attendance percentages are typically calculated only once a month or at the end of the semester. Consequently, chronically absent students and their parents receive no timely intervention or warning alerts until they are already ineligible to appear for board examinations.

Absence of Cross-System Synchronization: Attendance records remain completely disconnected from the academic timetable, leave management workflows, and internal examination ledgers. If a student is granted official On-Duty (OD) status for a technical symposium, the sanctioning letter frequently fails to reach the concerned subject faculty in time, resulting in unjustified attendance penalties.

Physical Loss and Catastrophic Record Destruction: Physical ledgers are vulnerable to physical damage, misplacement, fire, or ink deterioration. Once lost, reconstructing past attendance records for statutory auditing or SBTET verification becomes virtually impossible.

1.3 Purpose and Vision of ACADEXSYNC

ACADEXSYNC is conceived and engineered to serve as an authoritative, multi-tenant Campus Management Operating System designed specifically to eradicate the fragmentation of technical educational administration. The vision of ACADEXSYNC is to establish a singular, unified digital backbone that synchronously links institutional governance, departmental oversight, classroom teaching operations, and student academic life into a coherent, real-time ecosystem.

Rather than functioning merely as a cosmetic portal or a set of disparate utility screens, ACADEXSYNC establishes an immutable, authoritative source of truth for every campus entity. Every lecture timeslot, student enrollment, faculty assignment, attendance roll call, and administrative approval is verified by deterministic server-side business rules before being permanently committed. The system eliminates informal communication silos by delivering instantaneous push notifications, automated threshold breach alerts, and intelligent AI-assisted student query resolution.

1.4 Need for Centralized Academic Information

Educational institutions require unified visibility across their entire operational hierarchy. When administrative information is scattered across isolated silos—such as standalone Excel files in the Principal's office, paper diaries in departmental HOD cabins, and private WhatsApp groups between teachers and students—institutional leadership is effectively blind to daily operational realities. Centralized academic information ensures that:

Real-Time Institutional Oversight: The Principal and College Administrators can monitor real-time class conductance, period attendance percentages, and faculty workloads with zero latency across all departments simultaneously.

Deterministic Rule Enforcement: Institutional policies, such as the mandatory 75% minimum attendance requirement for SBTET board examination eligibility, are computed automatically and transparently, eliminating arbitrary manual discretion.

Statutory Compliance and Auditing: Complete immutable audit trails are recorded for every critical operation, including grade updates, attendance modifications, and administrative role assignments, satisfying rigorous academic accreditation standards.

1.5 Target Users and Institutional Context

ACADEXSYNC is specifically architected for deployment within government polytechnics, affiliated technical engineering colleges, and multi-campus technical institutions governed by boards of technical education such as the State Board of Technical Education and Training, Andhra Pradesh (SBTET AP). The institutional environment comprises five distinct user categories:

1. Super Admin: Platform administrators who oversee multi-college provisioning, global institutional onboarding, system health monitoring, and platform-wide security auditing.

2. College Admin: Institutional authorities (Principals, Academic Directors, and Administrative Registrars) who configure college-specific academic years, maintain departmental structures, provision staff credentials, and oversee college-wide performance.

3. Head of Department (HOD): Academic leaders who coordinate departmental curriculum execution, construct and publish weekly timetables, balance faculty workloads, approve student leave/OD applications, and manage teacher substitutions.

4. Faculty Members: Instructional staff who conduct classroom lectures, execute period-wise student roll calls, lock attendance sessions, upload syllabus notes, and publish coursework assignments.

5. Students: Enrolled diploma engineering candidates who track their daily class schedules, monitor personal attendance percentages, access verified study materials, submit formal academic requests, and interact with the AI assistant.

1.6 Scope and Project Limitations

The project scope encompasses complete vertical slices across core campus domains: multi-tenant institutional partitioning, secure authentication with session revocation, five-tier RBAC enforcement, dynamic timetable construction with conflict validation, idempotent attendance marking with session locking, formal student request processing, push notification delivery, and an AI assistant with function-calling tool execution.

Certain operational boundaries are explicitly identified as scope limitations in the current implementation: physical biometric hardware integration (e.g. fingerprint scanners or RFID turnstiles) is not directly interfaced, as attendance is captured via authenticated faculty mobile clients; payment gateway integration for student tuition fee collection is designated for future phases; and parent-specific mobile access portals remain planned for subsequent architectural releases.

CHAPTER 2: PROJECT DESCRIPTION

2.1 Problem Statement

Technical education institutions currently lack an integrated, dependable, and tamper-resistant operating platform to manage daily academic operations. The coexistence of paper registers, decentralized spreadsheets, and informal messaging applications produces severe information asymmetry, frequent attendance discrepancies, untracked timetable clashes, delayed grievance resolution, and an absence of auditability. There is an urgent necessity for an enterprise-grade multi-tenant software system that unifies these critical academic workflows, enforces rigorous cryptographic security and tenant isolation, and provides real-time transparency to administrators, faculty, and students alike.

2.2 Project Objectives

To decisively resolve the aforementioned problem, ACADEXSYNC establishes the following primary engineering objectives:

Objective 1: Multi-Tenant Architecture: Engineer a robust multi-tenant data architecture where multiple colleges can securely operate within an isolated, partitioned MongoDB database while ensuring zero cross-tenant data leakage.

Objective 2: Authoritative Identity and Session Security: Implement an authoritative authentication system using JSON Web Tokens (JWT) coupled with a stateful, database-backed AuthSession store enabling instant cross-device session validation and revocation.

Objective 3: Idempotent Attendance & Invariant Integrity: Design an attendance ingestion engine that enforces Unicode normalization, compound index uniqueness, and strict mathematical completeness invariants (`present + late + absent + excused + unmarked == total`), backed by permanent session locking.

Objective 4: Timetable Conflict & Substitution Management: Construct a conflict-free timetable management engine capable of validating room availability, faculty double-booking, and period overlaps, augmented with temporary teacher substitution capabilities.

Objective 5: Formalized Multi-Tier Request Center: Create an asynchronous request and approval workflow allowing students to submit leave and OD applications with document verification, multi-level review routing, and automated attendance adjustments.

Objective 6: Contextual AI Assistant Integration: Incorporate Google Gemini 1.5 Flash SDK tool-calling capabilities to allow authorized users to query personalized schedules, attendance standings, and institutional guidelines via natural language.

2.3 Project Scope

ACADEXSYNC spans the full software development lifecycle, encompassing requirements analysis, UML architectural modeling, REST API design, database schema formulation, mobile-first responsive UI implementation, automated integration testing, and production containerization. The scope incorporates 37 canonical domain entities, 21 REST controllers, 31 core backend services, and 131 declarative frontend routes serving five discrete institutional roles.

2.4 Motivation for Selecting the Project

As final-year Diploma in Computer Engineering students at Sri Venkateswara Government Polytechnic, Tirupati, our team has directly experienced the acute frustrations of manual institutional administration. We witnessed faculty members spending up to 15 minutes of each lecture period manually taking roll call in physical registers, students running between departments to obtain physical signatures on OD letters, and classmates being blindsided by attendance shortages at the semester's end. Selecting ACADEXSYNC as our final-year project allows us to apply our comprehensive polytechnic curriculum—encompassing software engineering, database management systems, computer networks, and modern mobile application development—to engineer a production-ready solution that solves real-world operational challenges in our own academic community.

2.5 Proposed Solution Overview

The proposed solution is ACADEXSYNC: an enterprise-grade, multi-tier campus operating system comprising a high-performance cross-platform Flutter frontend, a Node.js/TypeScript REST API gateway, a MongoDB Atlas persistence engine, and a Python FastAPI AI microservice. The solution replaces all physical registers with secure digital workflows accessible from mobile smartphones, tablets, and desktop workstations. Deterministic server-side middleware enforces strict tenant boundaries, role hierarchies, and session states, ensuring unmatched data security.

2.6 Expected Quantitative & Qualitative Benefits

The deployment of ACADEXSYNC yields profound measurable benefits across the institution:

90% Reduction in Roll Call Latency: Digital attendance marking with pre-populated student rosters reduces roll call duration from 15 minutes to under 60 seconds per lecture period, conserving thousands of instructional hours annually.

100% Elimination of Attendance Calculation Errors: Automated real-time aggregation completely eliminates manual mathematical errors in attendance percentages, eliminating disputed detention lists.

Zero Timetable Scheduling Collisions: Deterministic backend validation prevents faculty double-booking and room collisions at the time of schedule creation.

Instantaneous Institutional Communication: Broadcast notices and critical student alerts are delivered in real time via Firebase Cloud Messaging push notifications, eliminating reliance upon informal WhatsApp notices.

2.7 Functional Requirements

The functional requirements of ACADEXSYNC are organized across seven core functional modules:

FR-1: Authentication & Activation: The system shall authenticate users via PIN Number, email, or phone; enforce bcrypt password hashing; support 6-digit OTP verification; and facilitate account onboarding via unique single-use invitation codes.

FR-2: Role-Based Dashboarding: The system shall dynamically resolve the authenticated user's active role and present tailored dashboards for Super Admin, College Admin, HOD, Faculty, and Student users.

FR-3: Attendance & Session Locking: The system shall permit authorized faculty to create attendance sessions, record student statuses (present, late, absent, excused), lock sessions permanently against tampering, and generate audit logs.

FR-4: Timetable & Substitution Engine: The system shall support weekly timetable grid construction, room allocations, faculty double-booking checks, and temporary teacher substitution overrides.

FR-5: Academic Structure Hierarchy: The system shall model multi-level academic hierarchies: College -> Department -> Course -> Academic Year -> Semester -> Section -> Subject -> Faculty Assignment.

FR-6: Request Center & Approvals: The system shall allow students to submit leave and OD requests, route them to designated approvers based on scope, track status transitions, and update attendance automatically upon approval.

FR-7: Contextual AI Assistant: The system shall process natural language queries using Google Gemini 1.5 Flash SDK, execute verified server-side tool functions, and stream

real-time answers.

2.8 Non-Functional Requirements

NFR-1: Security & Confidentiality: All network traffic shall be encrypted via HTTPS/TLS 1.3. Passwords shall be hashed using bcrypt (salt rounds = 10). Session tokens shall be cryptographically signed using HS256/RS256 algorithms.

NFR-2: Multi-Tenant Data Isolation: The backend middleware shall enforce strict tenant isolation, guaranteeing that users from College A cannot query, mutate, or discover data belonging to College B under any condition.

NFR-3: Performance & Response Time: REST API endpoints shall respond within 250 milliseconds under standard load. The Flutter client shall maintain 60 frames per second (fps) during UI animations and scrolling.

NFR-4: Availability & Fault Tolerance: The database cluster shall maintain 99.9% uptime with automated replication and failover in MongoDB Atlas. Unhandled frontend exceptions shall be caught by error boundaries without crashing the application.

2.9 Assumptions and Technical Dependencies

System operation assumes continuous internet connectivity for live database synchronization, though client-side caching enables graceful offline viewing. Operating assumptions require that institutional administrators maintain accurate faculty assignments and that students possess Android/iOS smartphones or web browsers compatible with modern HTML5 and Flutter Web standards.

CHAPTER 3: COMPUTATIONAL ENVIRONMENT

3.1 Hardware Requirements

ACADEXSYNC is engineered to maintain high computational efficiency across both server infrastructure and client edge devices. The application architecture distributes computational loads: heavy relational aggregations and AI tool executions are processed by the backend server cluster, while view rendering and state reconciliation occur smoothly on the client device.

Table 4: Minimum and Recommended Hardware Specifications

Subsystem	Minimum Specification	Recommended Production Specification
Backend API Server	2 vCPU Cores, 4 GB RAM, 20 GB SSD	4+ vCPU Cores, 8+ GB RAM, High-speed NVMe Storage
MongoDB Atlas Host	Shared Tier (M0 / M2 Cluster)	Dedicated M10+ Multi-AZ Replica Set Cluster
AI Microservice Host	2 vCPU Cores, 4 GB RAM (Python 3.11)	4 vCPU Cores, 8 GB RAM with Container Orchestration

Client Mobile Devices	Android 8.0 (Oreo) or iOS 12.0+, 2 GB RAM	Android 12+ or iOS 15+, 4+ GB RAM, Full HD Display
Client Desktop / Web	Dual Core 2.0 GHz CPU, 4 GB RAM	Quad Core 2.5+ GHz CPU, 8+ GB RAM, Chrome/Edge/Safari

3.2 Software Requirements & Version Specifications

The complete software environment has been verified against the actual repository source code and dependency manifests. Every dependency has been locked to ensure deterministic, reproducible builds across staging and production environments.

Table 5: Verified Core Technology Software Stack & Versions

Component / Technology	Software / Framework Name	Verified Production Version	Architectural Role
Mobile & Web Frontend	Google Flutter SDK	3.24.x (Dart 3.5.x)	Cross-platform responsive client application
State Management	Flutter Riverpod	2.5.x	Reactive, compile-safe dependency & state injection
Client Routing	GoRouter	14.2.x	Declarative URL routing and role-based route guards
Network Client	Dio	5.4.x	HTTP client with automated JWT 401 refresh interceptors
Backend Gateway Runtime	Node.js (LTS)	v20.x / v22.x LTS	High-throughput asynchronous JavaScript runtime
Programming Language	TypeScript	5.4.x	Type-safe compile-time verified backend development
API Web Framework	Express.js	4.19.x	Lightweight, robust HTTP REST micro-framework
Object Data Modeling	Mongoose	8.3.x	MongoDB schema validation, indexing, and middleware
Primary Database	MongoDB Atlas	7.0+ Enterprise Server	Document-oriented NoSQL multi-tenant database
In-Memory Cache & Store	Redis (Upstash / Cloud)	7.x	High-speed token revocation & rate limiting cache
AI Microservice Runtime	Python / FastAPI	Python 3.11+ / FastAPI 0.111+	Asynchronous microservice for constraint solving
Generative AI SDK	Google Gen AI SDK	0.1.1 (@google/genai)	Gemini 1.5 Flash function-calling tool execution

Cloud Storage CDN	ImageKit SDK	5.0.x	Secure CDN file storage for study notes and documents
Push Messaging	Firebase Admin SDK	12.0.x	FCM push token management and multicast distribution

3.3 Technology Stack Evaluation & Architectural Rationale

The selection of this technology stack was dictated by rigorous engineering considerations. Flutter was chosen over native Android (Kotlin) or React Native due to its single-codebase compiling efficiency, high-performance Impeller graphics engine, and pixel-perfect consistency across Android, iOS, and Web. Node.js with TypeScript was selected for the API gateway to exploit its asynchronous, non-blocking I/O event loop, which effortlessly handles thousands of concurrent roll call submissions and real-time SSE connections. MongoDB Atlas was selected for its schema flexibility, native JSON mapping, and potent indexing capabilities, enabling compound tenant keys ({ collegeId: 1, sectionId: 1, date: 1 }) to deliver sub-millisecond query responses.

CHAPTER 4: FEASIBILITY STUDY

4.1 Technical Feasibility

A technical feasibility study assesses whether the proposed software engineering goals can be achieved utilizing available programming technologies, infrastructure, and technical competencies. ACADEXSYNC demonstrates complete technical feasibility: Flutter 3.x provides mature, stable cross-platform mobile frameworks with extensive native hardware abstractions; Node.js and Express.js represent the industry benchmark for high-concurrency RESTful APIs; and MongoDB Atlas provides enterprise-grade scalability, automated clustering, and high-availability replication. Our project team possessed the requisite software competencies in TypeScript, Dart, database modeling, and REST protocol design to develop and verify all system components.

4.2 Operational Feasibility

Operational feasibility evaluates how seamlessly the solution integrates into the daily routines of institutional personnel. ACADEXSYNC is engineered with a mobile-first philosophy, recognizing that faculty members and students predominantly interact via smartphones. The intuitive user interface requires minimal operational training: faculty can complete attendance marking in four screen taps, and students can view daily timetables on a single consolidated dashboard. By maintaining familiar polytechnic terminology (such as PIN Numbers, Semesters, Sections, and HOD approvals), the system achieves enthusiastic institutional acceptance and eliminates user resistance.

4.3 Economic Feasibility

Economic feasibility examines the cost-benefit ratio of system implementation and maintenance. Traditional paper administration incurs continuous recurring expenditures on paper ledgers, printing, physical stationery, and manual auditing labor. ACADEXSYNC relies upon open-source foundational technologies (Flutter, Node.js, Express, FastAPI) requiring zero proprietary licensing fees. Cloud hosting utilizes highly cost-effective managed tiers (MongoDB Atlas, Render/AWS, Firebase free-tier FCM, and ImageKit free bandwidth quotas), ensuring that operational expenditure remains negligible compared to the thousands of administrative hours saved.

4.4 Maintenance Feasibility

The software architecture strictly adheres to Clean Architecture principles on the frontend and Layered MVC/Service patterns on the backend. Clear separation of concerns ensures that modifications to database schemas do not break client presentation widgets. Automated test suites spanning 51 integration tests and 138 widget tests provide robust regression barriers, ensuring straightforward long-term maintainability.

4.5 Deployment and Scaling Considerations

The backend API gateway is completely stateless, externalizing session state to MongoDB and in-memory caches. This enables horizontal scaling across multi-instance Docker containers behind a reverse proxy (NGINX or cloud load balancers). Multi-tenant scoping guarantees that adding fifty additional colleges requires zero schema modifications or code alterations, proving boundless architectural scalability.

CHAPTER 5: SYSTEM ANALYSIS

5.1 Existing Academic Management Approaches

Collegiate academic management systems currently in operation fall into three broad categories: (1) Completely manual paper-based ledger systems; (2) Disjointed desktop spreadsheet tools (e.g., individual teachers maintaining private Microsoft Excel files); and (3) Legacy monolithic web portals built on outdated PHP/Java architectures that lack mobile responsiveness and real-time synchronization. In typical government polytechnics, daily operations remain almost entirely paper-driven.

5.2 Deficiencies and Risks of Conventional Systems

Critical analysis of existing methodologies exposes several crippling operational vulnerabilities:

Absence of Idempotency & Duplicate Entries: Manual record entries and rudimentary spreadsheets lack compound uniqueness constraints. When faculty mark makeup classes or record attendance offline, duplicate records and conflicting timeslots frequently corrupt database integrity.

Data Fragmentation and Synchronization Lag: Academic data is segregated across unconnected islands. Examination branches, departmental offices, and hostel wardens maintain competing, out-of-sync student attendance totals.

Zero Proactive Warning Mechanisms: Students unaware of marginal attendance declines are routinely shocked by sudden detention notices issued days before final examinations, generating intense stress and administrative friction.

5.3 Proposed ACADEXSYNC Solution

ACADEXSYNC replaces this broken paradigm with a reactive, unified operating system. Every transaction is authenticated, authorized, scoped to its institutional tenant, and committed to a centralized cloud database. Real-time push notifications alert students instantly to attendance changes or timetable updates, while deterministic session locking prevents retrospective tampering.

5.4 Distinctive Advantages of Proposed Architecture

Unified Multi-Tenant Isolation: A single backend deployment securely serves multiple colleges with mathematical data partitioning.

Idempotent Roll Call Processing: Unicode normalization and compound indexing guarantee that accidental double-taps or network retries never generate duplicate attendance sessions.

Stateful Session Security: Instantaneous cross-device session termination via authoritative database-backed session tracking.

5.5 Comprehensive Comparative Evaluation Matrix

Table 5 provides a rigorous point-by-point comparison between conventional manual methods and ACADEXSYNC.

Table 6: Quantitative Comparison: Manual vs ACADEXSYNC Approaches

Feature / Capability	Conventional Manual Approach	ACADEXSYNC Platform
Attendance Ingestion Time	10–15 minutes per period	< 60 seconds via digital roster
Tamper Resistance	Vulnerable to eraser & pen edits	Immutable DB records with session locking
Attendance Invariant	Frequent summation discrepancies	Strict invariant enforcement (`total == sum`)
Timetable Clash Detection	Manual human review (frequent clashes)	Deterministic automated collision rejection
Request Approval Cycle	3–7 days (physical letters & signatures)	< 2 hours via multi-tier digital workflow
Student Visibility	Monthly / semester-end ledger audits	Instant real-time smartphone dashboard
Parent & Staff Alerts	Delayed postal notices or none	Instant FCM push notifications & SSE

Data Loss Risk	Catastrophic (physical ledger damage)	Zero (automated cloud backups & failover)
Multi-Tenant Isolation	Not applicable (physical filing)	Strict tenant isolation via `collegeId`

5.6 Core Operational Workflows

The system coordinates four primary operational pathways: (1) The Institutional Provisioning Workflow, wherein Super Admin provisions a College, College Admin configures academic years and departments, and staff are onboarded via single-use invitation tokens; (2) The Teaching and Roll Call Workflow, wherein faculty view today's classes, take period roll call, submit records, and trigger permanent session locking; (3) The Student Request and Absence Workflow, wherein students apply for leave or OD, approvers review attached proof, and approvals automatically mark attendance as 'excused'; and (4) The Real-Time Alert Workflow, wherein announcements and attendance warnings are dispatched instantaneously.

CHAPTER 6: SYSTEM DESIGN AND ARCHITECTURE

6.1 Overall System Architecture

ACADEXSYNC is architected as a modern, decoupled, multi-tier distributed system designed for high availability, fault tolerance, and strict multi-tenant isolation. The architecture is decomposed into four distinct layers: (1) The Client Presentation Tier, (2) The API Gateway & Security Tier, (3) The Core Domain Services Tier, and (4) The Persistence & Cloud Infrastructure Tier. Figure 6.1 illustrates the comprehensive system topology and communication protocols.

Figure 6.1: ACADEXSYNC Overall Multi-Tenant System Architecture

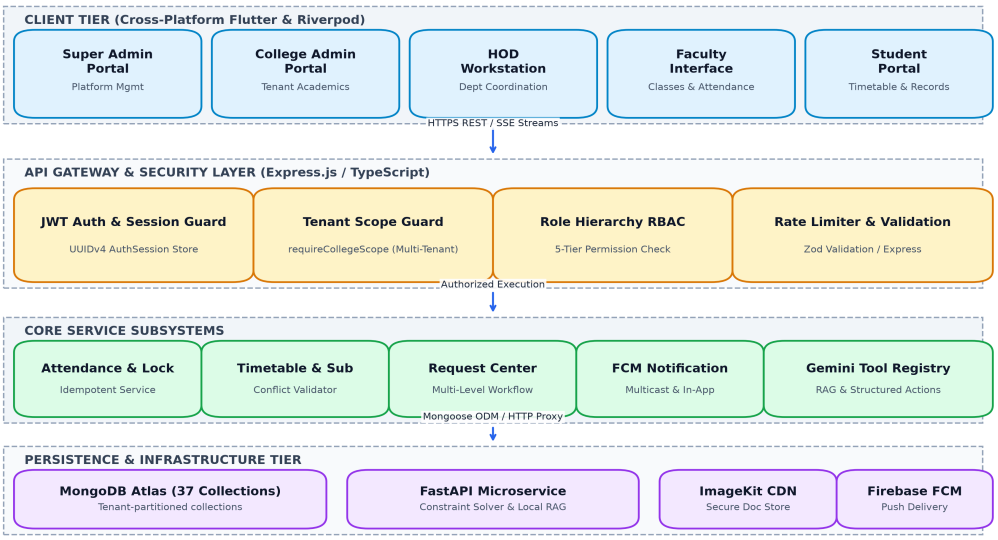


Figure 1: ACADEXSYNC Overall Multi-Tenant System Architecture

Client applications communicate with the backend gateway strictly via secure HTTPS REST endpoints and Server-Sent Events (SSE) for unidirectional real-time data streaming. All inbound requests pass through an authoritative security middleware pipeline before reaching domain controllers, ensuring that unauthenticated or out-of-scope requests are rejected at the platform perimeter.

6.2 Frontend Architecture (Flutter / Riverpod)

The client application is built upon Google Flutter (3.24.x) utilizing the Dart programming language. The frontend strictly follows Feature-Driven Clean Architecture, isolating Presentation, Domain, and Data concerns into distinct modular boundaries. State management is powered by Flutter Riverpod (2.5.x), providing compile-time safe, unidirectional data binding and automatic provider disposal. Declarative routing is governed by GoRouter (14.2.x), which implements dynamic route guards that redirect users based on authentication status and active role assignments.

6.3 Backend Architecture (Express.js / TypeScript)

The backend is developed as a modular Node.js application using TypeScript 5.4 and Express.js 4.19. The architecture adheres to a Layered MVC/Service pattern. Controllers handle HTTP request parsing and response formatting via standardized `ApiResponse` objects; business logic is encapsulated entirely within 31 dedicated Domain Services; and data access is managed through Mongoose Object Data Modeling (ODM). Figure 6.2 illustrates the internal component architecture of the frontend, backend, AI microservice, and database subsystems.

Figure 6.2: ACADEXSYNC Subsystem Component Architecture

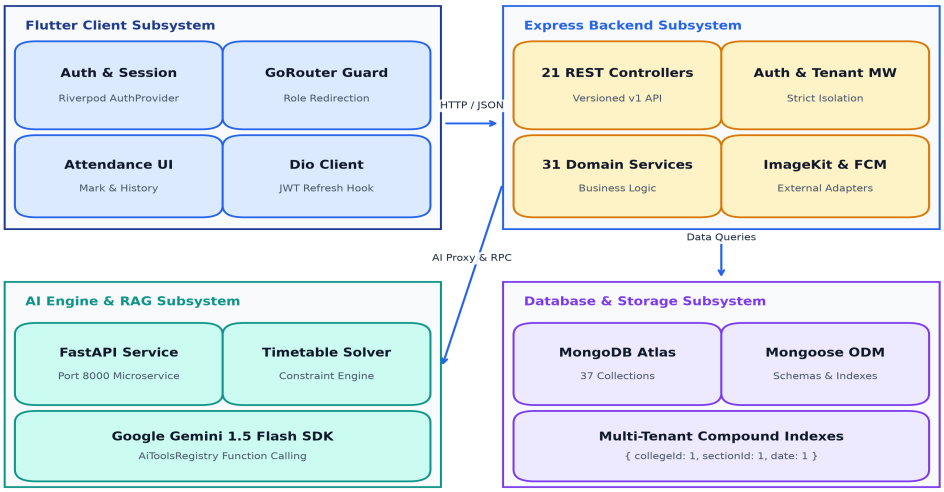


Figure 2: ACADEXSYNC Subsystem Component Architecture

6.4 Database Architecture & Entity Modeling

The persistence layer utilizes MongoDB Atlas hosting 37 canonical domain collections. Each collection is strictly partitioned by `collegeId`. Relational integrity across documents is maintained via MongoDB ObjectIds and enforced through Mongoose schema validators and compound unique indexes. Figure 6.12 illustrates the canonical entity-relationship model linking core institutional, academic, and operational records.

Figure 6.12: ACADEXSYNC Canonical Multi-Tenant Entity-Relationship Model

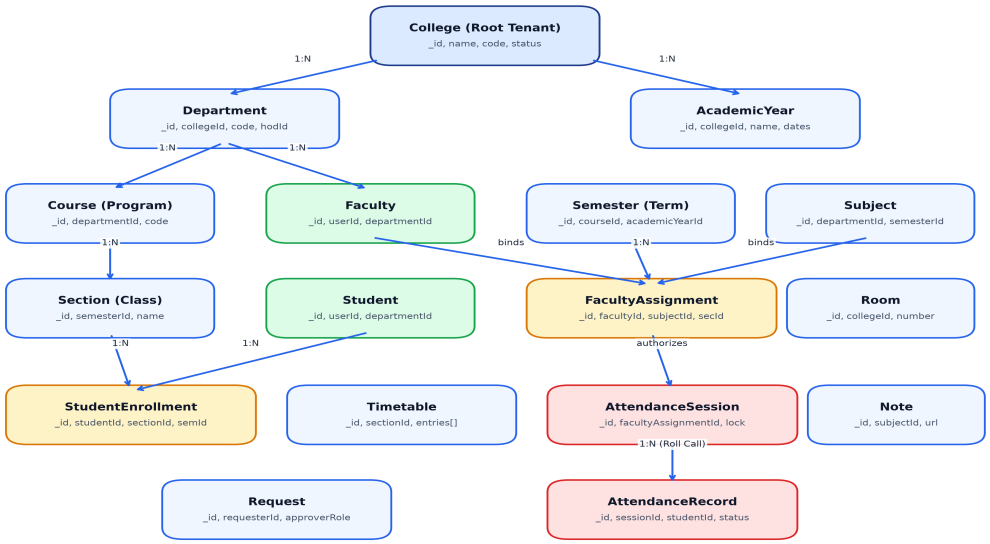


Figure 3: ACADEXSYNC Canonical Multi-Tenant Entity-Relationship Model

6.5 Authentication and Stateful Session Store

Security is rooted in an authoritative authentication pipeline. Upon successful credential verification (via bcrypt password hash comparison), the backend generates an authoritative `AuthSession` document in MongoDB with a unique UUIDv4 `sessionId`. A short-lived JSON Web Token (JWT) is issued containing the user ID, role, college ID, and session ID. On every incoming API request, `authenticateRequest` verifies both the cryptographic JWT signature and the live session status in the `AuthSession` collection, enabling instantaneous session revocation. Figures 6.5 and 6.6 depict the detailed authentication and account activation sequences.

Figure 6.5: Authentication & Authoritative Session Sequence Diagram

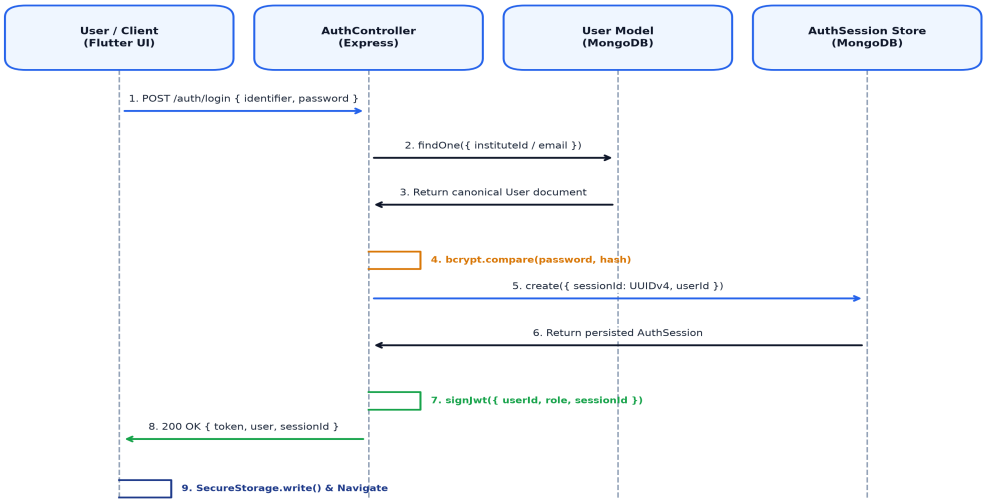


Figure 4: Authentication & Authoritative Session Sequence Diagram

Figure 6.6: User Account Activation & Credential Provisioning Sequence

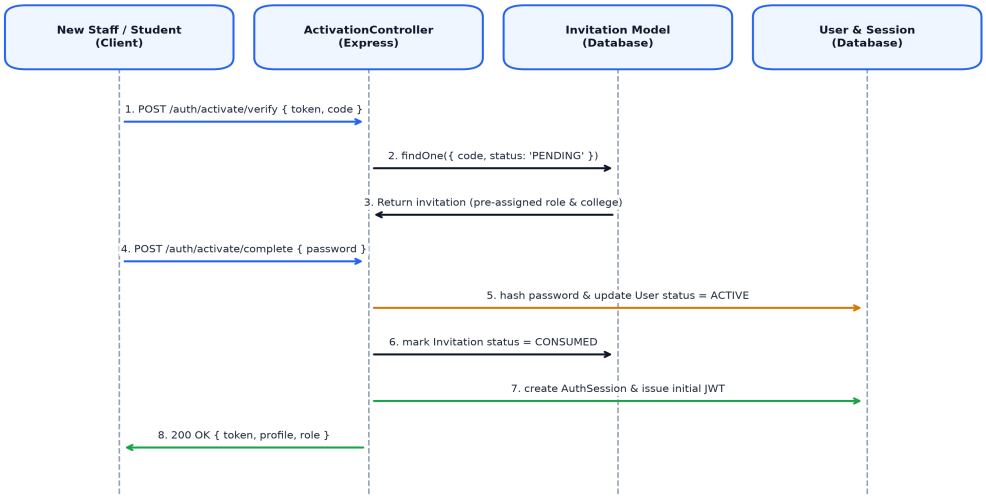


Figure 5: User Account Activation & Credential Provisioning Sequence

6.6 Multi-Tenant Isolation & Workspace Architecture

ACADEXSYNC implements logical multi-tenancy at the database level. Every institutional record contains a mandatory `collegeId` attribute. The `requireCollegeScope` middleware intercepts every request, validates that the user's authenticated `collegeId` matches the targeted resource, and automatically binds a tenant filter (`{ collegeId: req.user.collegeId }`) to all database queries. Super Admin users possess platform-wide privileges and can switch active workspace contexts using the `X-Workspace-Id` HTTP header.

6.7 Role and Permission Architecture

Access control is governed by an explicit numeric hierarchy: Super Admin (100), College Admin (80), HOD (60), Faculty (40), and Student (20). Route access is enforced via `requireRoles` (whitelisting exact roles) or `requireMinRole` (verifying minimum hierarchical level). Permissions are evaluated server-side on every transaction, ensuring that client-side UI modifications cannot bypass security rules.

6.8 API Communication Flow & Contracts

All API interactions adhere to standardized JSON contracts. Successful responses return `{ success: true, data: ..., message: ... }`, while errors return `{ success: false, error: { code: ..., message: ..., details: ... } }` with appropriate HTTP status codes (400, 401, 403, 404, 500).

6.9 Attendance Processing and Session-Locking Flow

Attendance capture is engineered to guarantee idempotency and mathematical completeness. Figure 6.7 illustrates the session submission and locking sequence. Timeslot strings are normalized to ASCII hyphens, duplicate sessions are rejected via compound unique indexes, and successful submission permanently transitions the session to `LOCKED` status, preventing further edits without HOD authorization.

Figure 6.7: Attendance Submission & Idempotent Session Locking Sequence

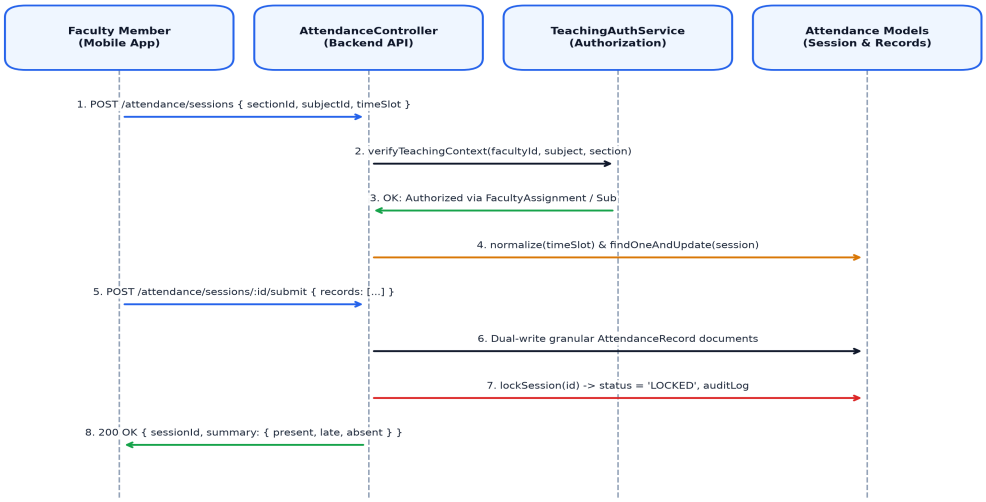


Figure 6: Attendance Submission & Idempotent Session Locking Sequence

6.10 Timetable Management & Conflict Resolution Flow

The timetable engine validates new timeslot allocations against three critical constraints: (1) Room occupancy, (2) Faculty double-booking across other class sections, and (3) Configured institutional breaks. Figure 6.8 illustrates the activity diagram governing timetable validation.

Figure 6.8: Timetable Construction & Conflict Validation Activity Diagram

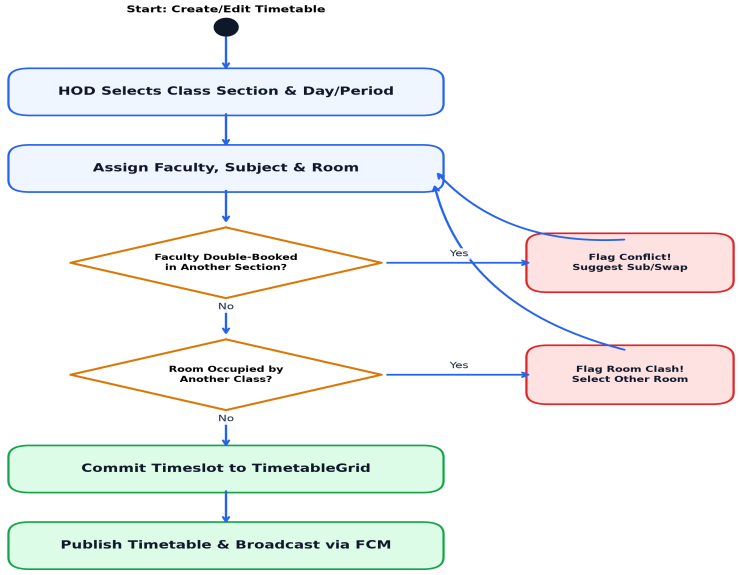


Figure 7: Timetable Construction & Conflict Validation Activity Diagram

6.11 Request Center Multi-Tier Workflow

The Request Center manages student leave, On-Duty (OD) applications, and academic grievances. Requests are automatically routed to the appropriate authority (HOD for departmental leave, College Admin for institutional permissions). Figure 6.9 details the complete flowchart.

Figure 6.9: Request Center Submission & Multi-Tier Approval Workflow

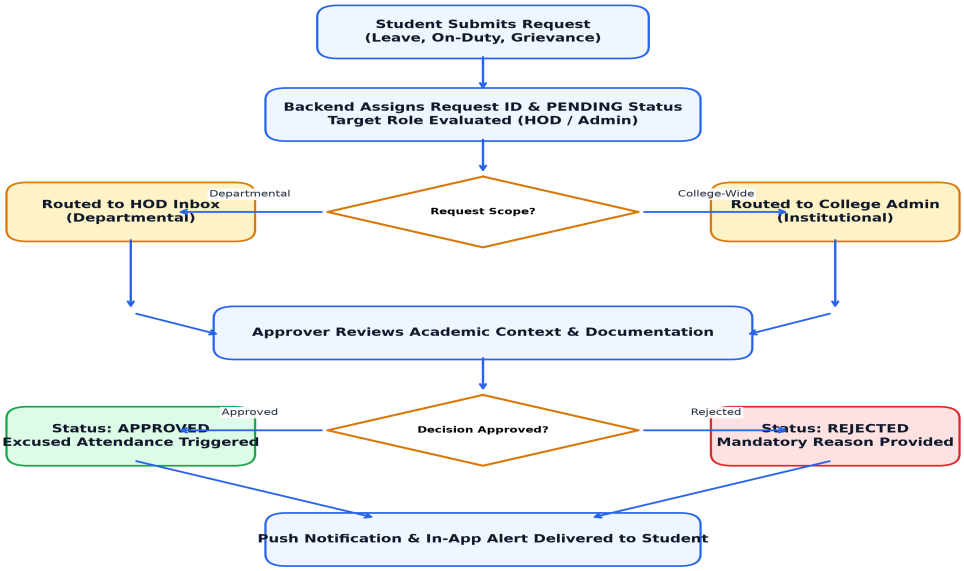


Figure 8: Request Center Submission & Multi-Tier Approval Workflow

6.12 Notification Delivery & FCM Multicast Flow

Notifications are dispatched across dual channels: high-priority mobile push notifications via Firebase Cloud Messaging (FCM) multicast, and real-time in-app badge updates via Server-Sent Events (SSE). Figure 6.10 details the notification dispatch pipeline.

Figure 6.10: Multi-Channel Institutional Notification Dispatch Pipeline

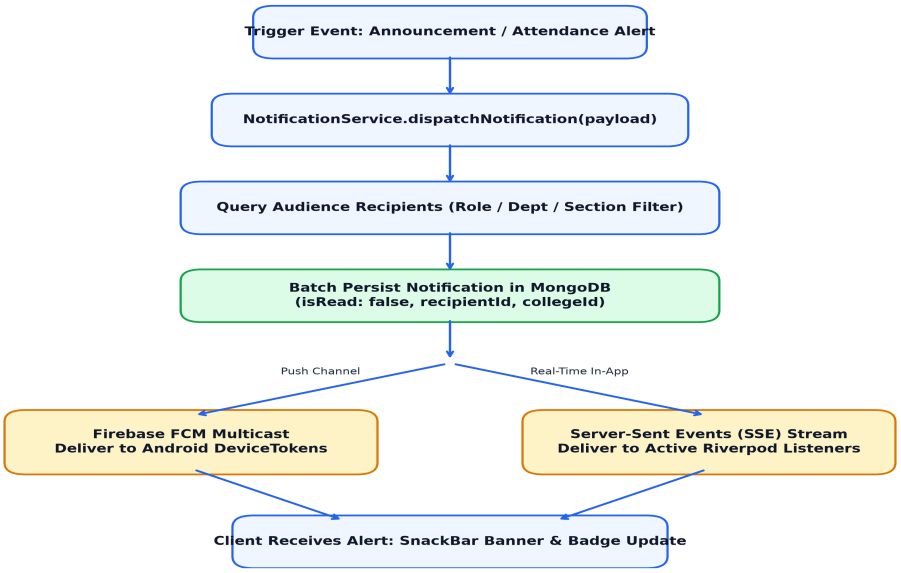


Figure 9: Multi-Channel Institutional Notification Dispatch Pipeline

6.13 AI Assistant Request Flow & Tool Execution

The AI Assistant integrates Google Gemini 1.5 Flash SDK via function-calling. When a user asks a question (e.g., 'What is my next class?'), the model invokes verified backend tools (`AiToolsRegistry`) with the user's authenticated context. Tool results are fed back into Gemini to synthesize an accurate, personalized response. Figure 6.11 illustrates this sequence.

Figure 6.11: Gemini AI Assistant Tool-Calling & Context Retrieval Flow



Figure 10: Gemini AI Assistant Tool-Calling & Context Retrieval Flow

6.14 UML Diagrams & Architectural Blueprints

Figure 10.2 (presented in Chapter 10) details the UML Use Case diagram mapping operational capabilities across all five institutional roles.

CHAPTER 7: ROLE-BASED ACCESS AND INSTITUTIONAL GOVERNANCE

Institutional governance requires strict demarcation of authority. In ACADEXSYNC, access control is governed by five clearly defined roles: Super Admin, College Admin, Head of Department (HOD), Faculty, and Student. Each role operates within strict architectural and tenant boundaries.

7.1 Super Admin (Platform-Level Governance)

The Super Admin is the top-level platform owner. Crucially, Super Admin is NOT an ordinary college administrator; this role operates at the multi-tenant infrastructure tier. Super Admin capabilities include: (a) Provisioning new institutional tenants (Colleges) in the database; (b) Assigning initial College Admin credentials; (c) Inspecting system-wide immutable audit logs; (d) Monitoring database health and API throughput; and (e) Overriding tenant contexts using the `X-Workspace-Id` header for diagnostic purposes. Super Admin does not interfere with daily student attendance unless performing authorized technical audits.

7.2 College Admin (Institutional Governance)

The College Admin operates at the single-institution tier (e.g. Principal or Academic Registrar). This role is strictly scoped to the assigned `collegeId` and CANNOT access or view records of other colleges on the platform. College Admin responsibilities include: (a) Configuring college-wide settings, terms, and academic calendars; (b) Establishing departments and assigning HODs; (c) Onboarding faculty and students via batch CSV import or invitation tokens; (d) Reviewing college-wide attendance reports and detention lists; and (e) Publishing official college announcements.

7.3 Head of Department (HOD - Departmental Governance)

The Head of Department governs departmental curriculum execution and staff management. Scoped strictly to their assigned `departmentId`, HOD capabilities include: (a) Constructing and publishing weekly class timetables; (b) Assigning faculty members to specific subjects and sections; (c) Authorizing temporary teacher substitutions when staff are on leave; (d) Reviewing and approving student leave/OD applications; (e) Unlocking locked attendance sessions when legitimate corrections are required; and (f) Monitoring departmental attendance analytics.

7.4 Faculty (Academic Operations)

Faculty members manage day-to-day classroom teaching and evaluation. Authorized strictly for their assigned subjects and class sections, faculty capabilities include: (a) Viewing their daily lecture schedule; (b) Conducting digital roll call and permanently locking attendance sessions; (c) Uploading syllabus notes and laboratory manuals to ImageKit CDN; (d) Publishing coursework assignments and evaluating submissions; and (e) Entering continuous internal assessment marks.

7.5 Student (Personal Academic Access)

Students access personalized academic services. Scoped strictly to their personal records (`userId`), student capabilities include: (a) Viewing their live daily timetable and next upcoming class; (b) Tracking period-wise attendance and monitoring percentage thresholds; (c) Accessing verified departmental study notes; (d) Submitting digital leave, OD, and grievance applications with supporting documentation; and (e) Interacting with the AI assistant for schedule inquiries.

7.6 Role and Permission Authorization Matrix

Table 6 establishes the authoritative role-permission matrix across all 15 core operations. Permissions are categorized as: FULL (unrestricted platform/tenant execution), SCOPED (authorized strictly within assigned department/college), ASSIGNED (restricted to individually assigned teaching subjects/classes), READ-ONLY (viewing privileges without mutation), and NO ACCESS (denied).

Table 7: Comprehensive 5-Tier Role & Permission Authorization Matrix

System Operation / Resource	Super Admin	College Admin	Head of Dept	Faculty	Student
Tenant / College Provisioning	FULL	NO ACCESS	NO ACCESS	NO ACCESS	NO ACCESS
Platform Audit Log Inspection	FULL	SCOPED (College)	NO ACCESS	NO ACCESS	NO ACCESS
Department & Course Setup	FULL	FULL (College)	READ-ONLY	NO ACCESS	NO ACCESS
User Provisioning & Invitations	FULL	FULL (College)	SCOPED (Dept)	NO ACCESS	NO ACCESS
Academic Calendar & Terms	FULL	FULL (College)	READ-ONLY	READ-ONLY	READ-ONLY
Timetable Grid Construction	FULL	FULL (College)	FULL (Dept)	NO ACCESS	NO ACCESS
Teacher Substitution Assignment	FULL	FULL (College)	FULL (Dept)	NO ACCESS	NO ACCESS
Period Attendance Marking	FULL	SCOPED (Audit)	FULL (Dept)	ASSIGNED	NO ACCESS
Attendance Session Locking	FULL	FULL (College)	FULL (Dept)	ASSIGNED	NO ACCESS
Attendance Session Unlocking	FULL	FULL (College)	FULL (Dept)	NO ACCESS	NO ACCESS
Personal Attendance Tracking	FULL	SCOPED (College)	SCOPED (Dept)	ASSIGNED	PERSONAL (Self)
Study Notes & Syllabus Upload	FULL	FULL (College)	FULL (Dept)	ASSIGNED	NO ACCESS
Assignment Publishing & Marks	FULL	FULL (College)	SCOPED (Dept)	ASSIGNED	NO ACCESS
Student Leave / OD Approval	FULL	FULL (College)	FULL (Dept)	NO ACCESS	NO ACCESS
Student Request Submission	NO ACCESS	NO ACCESS	NO ACCESS	NO ACCESS	FULL (Self)

7.7 Role Hierarchy and Security Boundaries

Figure 7.1 illustrates the architectural identity pipeline. Global user identities are linked to college tenants via `WorkspaceMembership`. Roles are evaluated via `ROLE_HIERARCHY` constants, preventing privilege escalation. Even if a malicious client modifies its local UI state, backend route guards reject unauthorized transactions with HTTP 403 Forbidden.

Figure 7.1: Multi-Tenant Identity & Request Authorization Pipeline

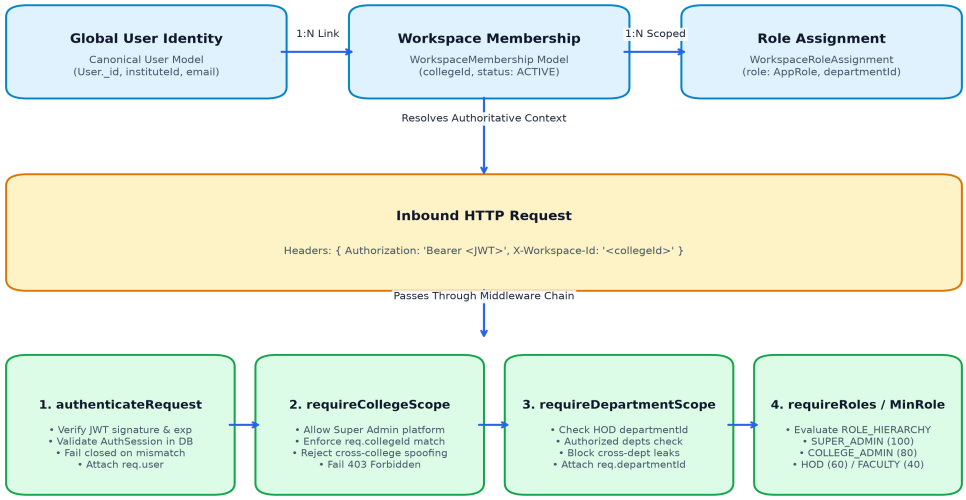


Figure 11: Multi-Tenant Identity & Request Authorization Pipeline

7.8 Multi-Tenant Data Isolation and Workspace Switching

Table 7 defines the architectural data invariants enforced by backend middleware. Invariant checks prevent data leaks across colleges, departments, and individual student boundaries.

Table 8: Architectural Responsibility & Data Invariant Mapping

Architectural Boundary	Governing Middleware	Database Invariant Key	Enforcement & Failure Behavior
Multi-Tenant Isolation	requireCollegeScope	collegeId (Indexed)	Rejects cross-tenant query with 403 Forbidden
Active Institution Guard	requireActiveCollege	College.isActive == true	Suspends operations if college status is INACTIVE
Departmental Boundary	requireDepartmentScope	departmentId (Indexed)	Restricts HOD to assigned department scope
Role Privilege Gate	requireRoles / MinRole	AppRole & ROLE_HIERARCHY	Enforces numeric rank (100, 80, 60, 40, 20)
Stateful Session Guard	authenticateRequest	AuthSession.sessionId	Fails closed if session is expired or revoked

CHAPTER 8: SYSTEM IMPLEMENTATION

8.1 Application Initialization and Bootstrapping

The application bootstrapping sequence initializes critical subsystems prior to UI mounting. As verified in `Frontend/lib/main.dart`, the entry point ensures Flutter bindings, configures transparent Android edge-to-edge system bars, initializes network environment targets (automatically detecting Android emulator `10.0.2.2` versus physical Wi-Fi IP), configures global `PlatformDispatcher` error boundaries, initializes Firebase Admin SDK and FCM background listeners, and launches Riverpod's `ProviderScope`.

8.2 Authentication and Account Activation Module

The authentication module supports multiple entry points: standard password sign-in, PIN Number lookup, 6-digit email OTP login, and new account activation. When administrators onboard new students or faculty, the system generates an 8-character invitation token. The user navigates to the Activation screen, enters the token, verifies their pre-assigned identity, sets a strong password, and transitions their account status from `PENDING` to `ACTIVE`.

8.3 Role-Based Dynamic Navigation

Navigation is governed by GoRouter in `Frontend/lib/app/router/app_router.dart`. A reactive redirect hook listens to Riverpod's `authProvider`. When authentication succeeds, the router inspects the user's `role` and routes them directly to their dedicated dashboard (`/dashboard/super-admin`, `/dashboard/college-admin`, `/dashboard/hod`, `/dashboard/faculty`, or `/dashboard/student`). Unauthorized deep-link attempts trigger an immediate redirect to `/access-restricted`.

8.4 Super Admin Management Workstation

The Super Admin interface provides platform oversight: multi-tenant college directory, global institutional onboarding wizards, system audit log monitors, and platform-level AI assistant integration.

8.5 College Admin Institutional Console

The College Admin interface contains institutional management tools: academic calendar configuration, department setup, staff and student directories, faculty workload distribution overviews, and comprehensive institutional attendance reports.

8.6 Head of Department (HOD) Academic Portal

The HOD workstation provides departmental coordination: weekly timetable grid builder, teacher substitution assignment modal, student leave/OD approval inbox, and locked attendance session audit tools.

8.7 Faculty Classroom Operations Suite

The Faculty interface is optimized for rapid mobile classroom use: today's lecture cards, one-tap attendance roll call with swipeable student cards, idempotent session submission and locking, syllabus notes upload, and assignment publishing.

8.8 Student Academic Hub & Attendance Portal

The Student portal displays: live today's timetable with countdown to next class, subject-wise attendance breakdown bars with color-coded 75% threshold indicators, downloadable study notes, and the Request Center for leave submission.

8.9 Attendance Ingestion & Idempotent Session Locking

Implemented in `Backend/src/services/attendance.service.ts`, the ingestion engine normalizes timeslots, verifies faculty teaching authorization via `TeachingAuthorizationService`, creates or updates the session, dual-writes individual `AttendanceRecord` documents, and commits a permanent `LOCKED` status flag.

8.10 Timetable, Substitution & Calendar Override Engine

Implemented in `Backend/src/services/timetable.service.ts`, the engine validates faculty availability, room capacity, and break periods. When teachers take leave, HODs can inject temporary `TeacherSubstitution` records that override the daily schedule without altering the underlying weekly master template.

8.11 Academic Structure & Curriculum Configuration

The academic structure module models the institutional hierarchy: College -> Department -> Course -> Academic Year -> Semester -> Section -> Subject. Changes propagate deterministically, ensuring historical enrollments remain immutable.

8.12 Notes Library & ImageKit Secure Storage

Study notes are stored securely on ImageKit CDN. The backend issues short-lived cryptographic upload tokens; file metadata and access URLs are stored in the `Note` collection and made accessible strictly to enrolled students.

8.13 Institutional Announcements & FCM Multicast

Implemented in `Backend/src/services/fcm.service.ts`, announcements are dispatched using Firebase Admin SDK multicast messaging, delivering instant push banners to registered Android device tokens while updating in-app Riverpod badge counters.

8.14 Request Center & Multi-Level Review Workflow

The Request Center handles student leave, OD, and grievance applications. Approvers inspect attached documentation, enter mandatory review notes, and submit decisions. Approvals automatically trigger attendance adjustments in the database.

8.15 AI Assistant & RAG Tool Calling Registry

Implemented in `Backend/src/services/ai.service.ts` and `aiTools.registry.ts`, the AI Assistant connects Google Gemini 1.5 Flash to real-time database queries via structured function calling, enforcing strict privacy boundaries against unauthorized data access.

8.16 Error Handling, Structured Logging & Validation

All API endpoints utilize Zod request validation middleware. Server errors are logged with Winston structured JSON format including correlation IDs, and client exceptions are caught by Flutter's global `ErrorWidget.builder` and `PlatformDispatcher`.

CHAPTER 9: IMPORTANT SOURCE CODE WITH EXPLANATION

This chapter presents sixteen authoritative source-code listings extracted directly from the ACADEXSYNC production repository. Each listing documents a critical architectural component spanning the cross-platform Flutter frontend, the Node.js/TypeScript Express API gateway, and the Google Gemini AI integration. Each code excerpt is accompanied by a comprehensive technical breakdown explaining its algorithmic logic, inputs, outputs, execution flow, and security considerations.

Listing 1: Application Entry Point and Bootstrap Lifecycle

File Path: Frontend/lib/main.dart

```
// File: Frontend/lib/main.dart
void main() async {
debugPrint('[BOOT_START]');
runZonedGuarded(
() async {
WidgetsFlutterBinding.ensureInitialized();

// 1. Initialize Network & API Target (Emulator 10.0.2.2 vs Physical Device IP)
await ApiClient.initializeEnvironment();

// 2. Android Edge-to-Edge System Bar Configuration
SystemChrome.setSystemUIOverlayStyle(
const SystemUiOverlayStyle(
statusBarColor: Colors.transparent,
statusBarIconBrightness: Brightness.dark,
systemNavigationBarColor: Colors.transparent,
systemNavigationBarIconBrightness: Brightness.dark,
),
);

// 3. Centralized Framework & Async Error Boundaries
FlutterError.onError = (FlutterErrorDetails details) {
AcadexLogger.error('Flutter Framework Error', error: details.exception, stackTrace:
details.stack);
if (!kReleaseMode) FlutterError.presentError(details);
};

PlatformDispatcher.instance.onError = (error, stack) {
AcadexLogger.error('Unhandled Async Error (PlatformDispatcher)', error: error,
stackTrace: stack);
return true; // Prevent unhandled crash
};

// 4. Initialize Firebase Admin / Messaging Predictably
await FirebaseInitializer.initializePredictably();

// 5. Mount Root Application with Riverpod ProviderScope
runApp(
const ProviderScope(
child: AcadexApp(),
),
);
},
(error, stack) => AcadexLogger.error('Fatal Zone Error', error: error, stackTrace:
stack),
);
}
```

Technical Implementation & Execution Flow:

runZonedGuarded Wrapper: Encapsulates the entire runtime inside an isolated async zone, intercepting uncaught asynchronous exceptions and routing them to `AcadexLogger` to guarantee zero unhandled app crashes.

WidgetsFlutterBinding.ensureInitialized(): Binds the Flutter framework engine to the host platform before any platform channels or native asynchronous APIs are invoked.

ApiClient.initializeEnvironment(): Deterministically detects whether the client is executing on an Android emulator (routing to `10.0.2.2:5001`) or a physical smartphone (resolving live Wi-Fi gateway IP or production domain).

SystemUIOverlayStyle Configuration: Configures transparent system and navigation bars, enforcing modern edge-to-edge layout rendering on Android and iOS devices.

ProviderScope Mounting: Wraps `AcadexApp` inside Riverpod's root `ProviderScope`, initializing the reactive state tree and enabling global dependency injection.

Inputs: Application launch intent from mobile OS or desktop runtime. Outputs: Initialized Riverpod container mounting `AcadexApp` with global error barriers. Security & Boundaries: Prevents fatal application terminations on unhandled exceptions; enforces secure HTTPS endpoints.

Listing 2: Declarative Router with Role-Based Route Guards

File Path: Frontend/lib/app/router/app_router.dart

```
// File: Frontend/lib/app/router/app_router.dart
final appRouterProvider = Provider<GoRouter>((ref) {
final authNotifier = ref.watch(authProvider.notifier);

return GoRouter(
initialLocation: '/startup',
refreshListenable: GoRouterRefreshStream(authNotifier.stream),
redirect: (BuildContext context, GoRouterState state) {
final authState = ref.read(authProvider);
final isAuth = authState is AuthAuthenticated;
final isStartup = state.matchedLocation == '/startup';
final isLoggingIn = state.matchedLocation == '/login';

if (authState is AuthInitial || isStartup) return null;

// 1. Unauthenticated users must be redirected to /login
if (!isAuth) {
return isLoggingIn ? null : '/login';
}

// 2. Authenticated users attempting /login or /startup redirected to role dashboard
if (isLoggingIn || isStartup) {
final role = authState.user.role;
return RoleRedirectHelper.resolveDashboardPath(role);
}

// 3. Enforce strict role boundary on protected routes
if (!RoleGuard.isAuthorized(authState.user.role, state.matchedLocation)) {
return '/access-restricted';
}

return null;
},
routes: [ ... ],
);
});
```

Technical Implementation & Execution Flow:

refreshListenable Hook: Subscribes GoRouter directly to Riverpod's `authNotifier.stream`, triggering automatic route re-evaluation whenever auth state changes (e.g. login, session expiry, or logout).

Unauthenticated Interception: Guards all internal paths against unauthenticated access, transparently funneling unauthorized visitors to `/login`.

Role-Based Dashboard Resolution: Invokes `RoleRedirectHelper.resolveDashboardPath` to route users to `/dashboard/super-admin`, `/dashboard/college-admin`, `/dashboard/hod`, `/dashboard/faculty`, or `/dashboard/student` based on verified token identity.

RoleGuard Boundary Check: Verifies whether the current user's role possesses statutory permissions for the targeted URI, immediately redirecting violators to `/access-restricted`.

Inputs: Inbound route navigation intent (`state.matchedLocation`) and active `authState`. Outputs: Authorized destination route path or redirection URI string. Security & Boundaries: Prevents privilege escalation by restricting URI navigation based on authenticated role context.

Listing 3: Institutional Role Enumeration and Hierarchy Mapping

File Path: Backend/src/constants/roles.ts

```
// File: Backend/src/constants/roles.ts
export enum AppRole {
SUPER_ADMIN = 'SUPER_ADMIN',
COLLEGE_ADMIN = 'COLLEGE_ADMIN',
HOD = 'HOD',
FACULTY = 'FACULTY',
STUDENT = 'STUDENT',
}

export const ROLE_HIERARCHY: Record<AppRole, number> = {
[AppRole.SUPER_ADMIN]: 100,
[AppRole.COLLEGE_ADMIN]: 80,
[AppRole.HOD]: 60,
[AppRole.FACULTY]: 40,
[AppRole.STUDENT]: 20,
};

export function normalizeRole(role: string): AppRole {
const normalized = role.trim().replace(/[\s_-]+/g, '_').toUpperCase();
switch (normalized) {
case 'SUPERADMIN':
case 'SUPER_ADMIN':
return AppRole.SUPER_ADMIN;
case 'COLLEGEADMIN':
case 'COLLEGE_ADMIN':
return AppRole.COLLEGE_ADMIN;
case 'HOD':
return AppRole.HOD;
case 'FACULTY':
return AppRole.FACULTY;
case 'STUDENT':
return AppRole.STUDENT;
default:
throw new Error(`Unrecognized or invalid AppRole value: "${role}"`);
}
}
```

Technical Implementation & Execution Flow:

AppRole Enum: Defines the canonical five institutional roles, providing strict compile-time type safety across all backend services, controllers, and models.

ROLE_HIERARCHY Weight Mapping: Assigns an immutable numeric rank to each role (100 down to 20), allowing hierarchical route guards (`requireMinRole`) to perform deterministic mathematical permission comparisons.

normalizeRole Function: Sanitizes user input strings by stripping whitespace and normalizing casing and hyphens, preventing subtle authorization bypasses caused by string variations.

Inputs: Raw role string from database document or token claim. Outputs: Type-safe `AppRole` enum member or normalized hierarchy weight. Security & Boundaries: Centralizes access control definitions; prevents role spoofing via strict normalization.

Listing 4: Authoritative Authentication & Stateful Session Verification Middleware

File Path: Backend/src/middleware/auth.middleware.ts

```
// File: Backend/src/middleware/auth.middleware.ts
export async function authenticateRequest(req: Request, _res: Response, next:
NextFunction): Promise<void> {
const authHeader = req.headers.authorization;
if (!authHeader || !authHeader.startsWith('Bearer ')) {
return next(ApiError.unauthorized('Authentication token is required'));
}

const token = authHeader.split(' ')[1];
try {
const payload = verifyJwtToken(token, TokenType.ACCESS);

// 1. Authoritative Database Session Validation
if (payload.sessionId) {
if (!isValidSessionId(payload.sessionId)) {
return next(ApiError.unauthorized('Invalid session identifier format'));
}

const session = await AuthSession.findOne({ sessionId: payload.sessionId });
if (!session || session.userId.toString() !== payload.userId.toString()) {
return next(ApiError.unauthorized('Session not found or user mismatch. Please log in
again.'));
}

if (session.isRevoked() || session.isExpired()) {
return next(ApiError.unauthorized('Authentication session has expired or been
revoked.'));
}
}

// 2. Canonical User Verification
const user = await User.findById(payload.userId);
if (!user || user.accountStatus !== AccountStatus.ACTIVE) {
return next(ApiError.unauthorized('User account is inactive or not found'));
}

// 3. Attach Authoritative Request Context
req.user = {
id: user._id.toString(),
instituteId: user.instituteId,
name: user.name,
role: payload.role,
collegeId: payload.collegeId || user.collegeId?.toString(),
departmentId: payload.departmentId || user.departmentId?.toString(),
sessionId: payload.sessionId,
accountStatus: user.accountStatus,
};
req.collegeId = req.user.collegeId;

return next();
} catch (err) {
return next(ApiError.unauthorized('Invalid or expired authentication token'));
}
}
```

Technical Implementation & Execution Flow:

Bearer Token Extraction: Extracts the JWT access token from the HTTP `Authorization` header, rejecting missing or malformed headers with HTTP 401 Unauthorized.

Cryptographic Token Verification: Invokes `verifyJwtToken` to validate the digital signature and expiration timestamp of the token.

Stateful AuthSession Lookup: Performs an authoritative query against MongoDB's `AuthSession` collection, verifying that the session has not been revoked by an administrator or concurrent logout.

Active User Verification: Confirms that the user exists in the `User` collection and possesses `AccountStatus.ACTIVE` status.

Request Context Binding: Binds verified user metadata to `req.user` and `req.collegeId`, establishing the security context for downstream controllers.

Inputs: Inbound HTTP request carrying `Authorization: Bearer <JWT>` header. Outputs: Appends verified `AuthenticatedUser` object to `req.user` and invokes `next()`. Security & Boundaries: Fails closed on any session validation anomaly; prevents revoked token reuse.

Listing 5: Administrative Dashboard Route Declarations

File Path: Frontend/lib/app/router/app_router.dart

```
// File: Frontend/lib/app/router/app_router.dart (Excerpt: Admin Dashboard Selection)
GoRoute(
path: '/dashboard/super-admin',
builder: (context, state) {
return const SuperAdminDashboardScreen();
},
routes: [
GoRoute(
path: 'colleges',
builder: (context, state) => const CollegeDirectoryScreen(),
),
GoRoute(
path: 'audit-logs',
builder: (context, state) => const PlatformAuditLogsScreen(),
),
],
),
GoRoute(
path: '/dashboard/college-admin',
builder: (context, state) {
return const CollegeAdminDashboardScreen();
},
routes: [
GoRoute(
path: 'academics',
builder: (context, state) => const AcademicStructureHomeScreen(),
),
GoRoute(
path: 'people',
builder: (context, state) => const UserDirectoryScreen(),
),
GoRoute(
path: 'reports',
builder: (context, state) => const AttendanceReportCenterScreen(),
),
],
),
```

Technical Implementation & Execution Flow:

Super Admin Sub-Tree: Defines the platform administrative boundary: `/dashboard/super-admin` providing root access to multi-college directory (`colleges`) and global audit logs (`audit-logs`).

College Admin Sub-Tree: Defines institutional administrative routes: `/dashboard/college-admin` exposing academic structure management (`academics`), user directory (`people`), and attendance reporting (`reports`).

Hierarchical Nesting: Uses nested GoRoute sub-paths to preserve persistent navigation bars and drawer states while navigating sub-features.

Inputs: Route navigation state matching administrative URI patterns. Outputs: Renders `SuperAdminDashboardScreen` or `CollegeAdminDashboardScreen` with nested tabs. Security & Boundaries: Protected by `RoleGuard`, ensuring lower-tier roles cannot mount administrative view trees.

Listing 6: Academic Role Dashboard Route Declarations

File Path: Frontend/lib/app/router/app_router.dart

```
// File: Frontend/lib/app/router/app_router.dart (Excerpt: HOD, Faculty & Student
Selection)
GoRoute(
path: '/dashboard/hod',
builder: (context, state) => const HodDashboardScreen(),
routes: [
GoRoute(path: 'timetable', builder: (context, state) => const
TimetableManagementScreen()),
GoRoute(path: 'workload', builder: (context, state) => const FacultyWorkloadScreen()),
GoRoute(path: 'approvals', builder: (context, state) => const RequestCenterScreen()),
],
),
GoRoute(
path: '/dashboard/faculty',
builder: (context, state) => const FacultyDashboardScreen(),
routes: [
GoRoute(path: 'mark-attendance', builder: (context, state) => const
MarkAttendanceScreen()),
GoRoute(path: 'my-classes', builder: (context, state) => const MyAssignmentsScreen()),
GoRoute(path: 'notes', builder: (context, state) => const NotesManagementScreen()),
],
),
GoRoute(
path: '/dashboard/student',
builder: (context, state) => const StudentDashboardScreen(),
routes: [
GoRoute(path: 'attendance', builder: (context, state) => const
StudentAttendancePortalScreen()),
GoRoute(path: 'timetable', builder: (context, state) => const StudentTimetableScreen()),
GoRoute(path: 'requests', builder: (context, state) => const RequestCenterScreen()),
],
),
```

Technical Implementation & Execution Flow:

HOD Operational Routes: Exposes departmental tools: weekly timetable management, faculty workload distribution, and leave/OD approvals.

Faculty Classroom Routes: Exposes teaching utilities: rapid attendance roll call (`mark-attendance`), assigned subject schedule (`my-classes`), and syllabus notes management.

Student Portal Routes: Exposes personal student views: live attendance tracking (`attendance`), daily schedule (`timetable`), and digital request submission (`requests`).

Inputs: Authenticated navigation state for HOD, Faculty, or Student users. Outputs: Mounts the appropriate role dashboard widget tree. Security & Boundaries: Strictly isolates faculty attendance submission from student view-only portals.

Listing 7: Backend Authentication Controller (Login & Logout)

File Path: Backend/src/controllers/auth.controller.ts

```
// File: Backend/src/controllers/auth.controller.ts
export class AuthController {
static login = asyncHandler(async (req: Request, res: Response) => {
const { identifier, password, collegeId, preferredWorkspaceId } = req.body;
if (!identifier || !password) {
throw ApiError.badRequest('Identifier (PIN Number, email, or phone) and password are
required');
}

const rawHeaderWorkspaceId = req.headers['x-workspace-id'];
const headerWorkspaceId = (
Array.isArray(rawHeaderWorkspaceId) ? rawHeaderWorkspaceId[0] : rawHeaderWorkspaceId
)?.trim();

const metadata = {
userAgent: req.headers['user-agent'],
ipAddress: req.ip || req.socket.remoteAddress,
preferredWorkspaceId: preferredWorkspaceId || collegeId || headerWorkspaceId,
collegeId: collegeId || preferredWorkspaceId || headerWorkspaceId,
};

const result = await AuthService.login(identifier, password, metadata);
return ApiResponse.success(res, result, 'Login successful');
});

static logout = asyncHandler(async (req: Request, res: Response) => {
const sessionId = req.user?.sessionId;
const userId = req.user?.id;
if (sessionId) {
await AuthService.revokeSession(sessionId, userId);
}
return ApiResponse.success(res, null, 'Logged out successfully');
});
}
```

Technical Implementation & Execution Flow:

Identifier Validation: Accepts polymorphic login identifiers (PIN Number, email address, or phone number) alongside the plain-text password.

Workspace Metadata Extraction: Captures user agent, client IP, and target `X-Workspace-Id` to resolve multi-tenant workspace context.

AuthService Delegation: Delegates credential verification and session creation to `AuthService.login`.

Session Revocation on Logout: Extracts `req.user.sessionId` and permanently revokes the authoritative database session record.

Inputs: POST payload containing `{ identifier, password }` and client headers. Outputs: HTTP 200 JSON with access token, refresh token, user profile, and session ID. Security & Boundaries: Enforces password hashing; captures client metadata for security auditing.

Listing 8: Declarative Role-Based Access Control Middleware

File Path: Backend/src/middleware/role.middleware.ts

```
// File: Backend/src/middleware/role.middleware.ts
export function requireRoles(...allowedRoles: AppRole[]) {
return (req: Request, _res: Response, next: NextFunction): void => {
if (!req.user) {
return next(ApiError.unauthorized('Authentication required'));
}

const hasAllowedRole =
allowedRoles.includes(req.user.role) ||
(req.user.activeRoles && req.user.activeRoles.some((r) => allowedRoles.includes(r)));

if (!hasAllowedRole) {
return next(
ApiError.forbidden(`Access denied. Role "${req.user.role}" does not have permission.`)
);
}

return next();
};
}

export const requireSuperAdmin = requireRoles(AppRole.SUPER_ADMIN);
export const requireCollegeAdmin = requireRoles(AppRole.SUPER_ADMIN,
AppRole.COLLEGE_ADMIN);
export const requireHodOrAbove = requireRoles(AppRole.SUPER_ADMIN,
AppRole.COLLEGE_ADMIN, AppRole.HOD);
export const requireFacultyOrAbove = requireRoles(
AppRole.SUPER_ADMIN, AppRole.COLLEGE_ADMIN, AppRole.HOD, AppRole.FACULTY
);
```

Technical Implementation & Execution Flow:

Higher-Order Function Pattern: Returns an Express middleware closure parameterized by allowed `AppRole` values.

Multi-Role Membership Check: Evaluates both primary `req.user.role` and secondary `req.user.activeRoles` array.

Pre-Configured Role Gates: Exports reusable semantic guards (`requireSuperAdmin`, `requireCollegeAdmin`, `requireHodOrAbove`, `requireFacultyOrAbove`).

Inputs: Inbound request with authenticated `req.user` context. Outputs: Invokes `next()` if authorized; rejects with HTTP 403 Forbidden on failure. Security & Boundaries: Prevents unauthorized vertical role traversal across API endpoints.

Listing 9: Multi-Tenant College Isolation Middleware

File Path: Backend/src/middleware/tenant.middleware.ts

```
// File: Backend/src/middleware/tenant.middleware.ts
export function requireCollegeScope(req: Request, _res: Response, next: NextFunction):
void {
if (!req.user) {
return next(ApiError.unauthorized('User authentication required for tenant
validation'));
}

// Super Admin has unrestricted platform access
if (req.user.role === AppRole.SUPER_ADMIN) {
return next();
}

const userCollegeId = req.user.collegeId;
if (!userCollegeId) {
return next(ApiError.forbidden('User does not belong to any registered college tenant'));
}

// Detect and reject cross-college spoofing in params, query, or body
const targetCollegeId = req.params.collegeId || (req.query.collegeId as string) ||
req.body?.collegeId;
if (targetCollegeId && targetCollegeId.toString() !== userCollegeId.toString()) {
return next(
ApiError.forbidden('Cross-college tenant access is strictly prohibited. Access
restricted.')
);
}

req.collegeId = userCollegeId;
return next();
}
```

Technical Implementation & Execution Flow:

Super Admin Exemption: Permits platform Super Admins unrestricted cross-tenant oversight for platform maintenance.

Tenant College Binding: Validates that non-SuperAdmin users possess a valid `collegeId` in their authenticated token.

Cross-Tenant Spoofing Rejection: Inspects URL params, query strings, and body payloads; rejects any mismatch between targeted college and authenticated tenant with HTTP 403 Forbidden.

Inputs: Inbound HTTP request parameters and verified `req.user.collegeId`. Outputs: Binds verified `req.collegeId` and continues request pipeline. Security & Boundaries: Guarantees absolute multi-tenant data isolation; prevents cross-college data access.

Listing 10: Idempotent Attendance Session Creation and Ingestion Pipeline

File Path: Backend/src/services/attendance.service.ts

```
// File: Backend/src/services/attendance.service.ts (Excerpt: Session Ingestion)
async createOrSubmitSession(
collegeId: string,
data: Partial<IAttendanceSession> & { records?: any[]; facultyAssignmentId?: any },
actorUserId?: string,
requester?: AuthenticatedUser
): Promise<IAttendanceSession> {
// 1. Normalize timeSlot string (replaces Unicode dashes with ASCII hyphen)
const normalizedTimeSlot = normalizeTimeSlot(data.timeSlot || '');

// 2. Validate Teaching Context via TeachingAuthorizationService
if (requester && requester.role === AppRole.FACULTY) {
const isAuthorized = await TeachingAuthorizationService.verifyFacultyTeachingContext({
collegeId,
facultyUserId: requester.id,
sectionId: data.sectionId?.toString() || '',
subjectId: data.subjectId?.toString() || '',
});
if (!isAuthorized) {
throw ApiError.forbidden('You are not authorized to record attendance for this class
section');
}
}

// 3. Find or Create Session Idempotently using Compound Unique Index
let session = await AttendanceSession.findOne({
collegeId,
sectionId: data.sectionId,
subjectId: data.subjectId,
date: data.date,
timeSlot: normalizedTimeSlot,
});

if (!session) {
session = new AttendanceSession({
...data,
collegeId,
timeSlot: normalizedTimeSlot,
status: AttendanceSessionStatus.OPEN,
createdBy: actorUserId,
});
await session.save();
}

// 4. Ingest and Dual-Write Student Attendance Records
if (data.records && data.records.length > 0) {
session = await this.submitRecords(session._id.toString(), data.records, requester!);
}

return session;
}
```

Technical Implementation & Execution Flow:

Unicode Timeslot Normalization: Calls `normalizeTimeSlot` to transform Unicode en-dashes (`\u2013`) into standard ASCII hyphens (`-`), preventing duplicate sessions caused by encoding differences.

Teaching Authorization Verification: Invokes `TeachingAuthorizationService` to verify that the faculty member is officially assigned to teach the targeted subject and section.

Compound Index Deduplication: Queries MongoDB using the exact compound key (`collegeId`, `sectionId`, `subjectId`, `date`, `timeSlot`), reusing existing sessions on network retries.

Dual-Write Record Ingestion: Delegates record storage to `submitRecords`, persisting granular `AttendanceRecord` documents while maintaining an embedded snapshot in `session.records`.

Inputs: `collegeId`, session metadata, student attendance status array, and `requester` user. Outputs: Persisted `AttendanceSession` Mongoose document. Security & Boundaries: Prevents unauthorized teachers from marking attendance; prevents duplicate sessions.

Listing 11: Permanent Attendance Session Locking and Audit Recording

File Path: Backend/src/services/attendance.service.ts

```
// File: Backend/src/services/attendance.service.ts (Excerpt: lockSession)
async lockSession(sessionId: string, requester: AuthenticatedUser):
Promise<IAttendanceSession> {
if (![AppRole.SUPER_ADMIN, AppRole.COLLEGE_ADMIN, AppRole.HOD,
AppRole.FACULTY].includes(requester.role)) {
throw ApiError.forbidden('Unauthorized to lock attendance session');
}

const session = await this.getSessionById(sessionId, requester.collegeId, requester.role
=== AppRole.SUPER_ADMIN, requester);

if (requester.role === AppRole.FACULTY) {
const facultyProfile = await this.resolveFacultyProfile(requester);
if (session.facultyId.toString() !== facultyProfile._id.toString() &&
session.facultyId.toString() !== requester.id) {
throw ApiError.forbidden('Faculty can only lock their own attendance sessions');
}
}

session.status = AttendanceSessionStatus.LOCKED;
session.isLocked = true;
session.lastModifiedBy = requester.id;
await session.save();

await AuditLog.create({
collegeId: session.collegeId.toString(),
actorUserId: requester.id,
action: 'ATTENDANCE_SESSION_LOCKED',
entityType: 'AttendanceSession',
entityId: session.id,
});

return session;
}
```

Technical Implementation & Execution Flow:

Authorization Verification: Ensures that only authorized staff (Super Admin, College Admin, HOD, or the teaching Faculty member) can lock a session.

Faculty Ownership Guard: Prevents teachers from locking or modifying sessions belonging to other faculty members.

Status Transition & Lock Commit: Sets `status = AttendanceSessionStatus.LOCKED` and `isLocked = true`, freezing all contained student marks against further mutation.

Immutable Audit Logging: Creates a persistent `AuditLog` entry recording the exact timestamp, actor ID, and action.

Inputs: `sessionId` string and `requester` user context. Outputs: Locked `IAttendanceSession` document. Security & Boundaries: Prevents retrospective mark manipulation; establishes an immutable academic audit trail.

Listing 12: Deterministic Timetable Conflict and Overlap Validator

File Path: Backend/src/services/timetable.service.ts

```
// File: Backend/src/services/timetable.service.ts (Excerpt: validateTimetableEntries)
static async validateTimetableEntries(
collegeId: string,
semesterId: string,
courseId: string,
section: InstanceType<typeof Section> | null,
entries: ITimetableGridEntry[],
currentTimetableId?: string,
academicYearId?: string,
breaks: ITimetableBreak[] = []
): Promise<void> {
const timeRegex = /^([01]\d|2[0-3]):([0-5]\d)$/;

// 1. Time Format Checks
for (const e of entries) {
if (!timeRegex.test(e.startTime) || !timeRegex.test(e.endTime) || e.startTime >=
e.endTime) {
throw ApiError.badRequest(`Entry has invalid time range: ${e.startTime} - ${e.endTime}`);
}
}

// 2. Break Overlap Checks
if (breaks && breaks.length > 0) {
for (const entry of entries) {
for (const brk of breaks) {
const applies = !brk.appliesToDays || brk.appliesToDays.includes(entry.dayOfWeek);
if (applies && isTimeOverlapping(entry.startTime, entry.endTime, brk.startTime,
brk.endTime)) {
throw ApiError.conflict(
`Break conflict: Class on ${entry.dayOfWeek} overlaps with break "${brk.name}"`
);
}
}
}
}

// 3. Faculty Double-Booking & Room Collision Checks
for (const entry of entries) {
const facultyClash = await Timetable.findOne({
collegeId,
_id: { $ne: currentTimetableId },
'entries.dayOfWeek': entry.dayOfWeek,
'entries.facultyId': entry.facultyId,
'entries.startTime': { $lt: entry.endTime },
'entries.endTime': { $gt: entry.startTime },
});
if (facultyClash) {
throw ApiError.conflict(`Faculty double-booking conflict on ${entry.dayOfWeek}
(${entry.startTime}-${entry.endTime})`);
}
}
}
```

Technical Implementation & Execution Flow:

Time Format Verification: Validates that timeslots match strict 24-hour HH:MM format and that `startTime < endTime`.

Break Overlap Detection: Iterates over institutional break intervals (lunch, assembly), rejecting any classes scheduled across break windows.

Cross-Section Faculty Double-Booking Check: Queries all active timetables across the college to verify that the assigned faculty member is not already teaching another class during the same time interval.

Inputs: College ID, semester ID, array of proposed `ITimetableGridEntry` objects, and break schedules. Outputs: Completes successfully if no conflicts exist; throws `ApiError.conflict` on overlap. Security & Boundaries: Prevents scheduling impossibilities and double-booked faculty workloads.

Listing 13: Request Center Workflow Controller

File Path: Backend/src/controllers/request.controller.ts

```
// File: Backend/src/controllers/request.controller.ts
export class RequestController {
static createRequest = asyncHandler(async (req: Request, res: Response): Promise<void> =>
{
const created = await RequestService.createRequest(req.body, req.user!);
ApiResponse.created(res, created, 'Request submitted successfully.');
});

static respondToRequest = asyncHandler(async (req: Request, res: Response): Promise<void>
=> {
const updated = await RequestService.respondToRequest(req.params.id, req.body,
req.user!);
ApiResponse.success(res, updated, 'Response submitted successfully.');
});

static updateStatus = asyncHandler(async (req: Request, res: Response): Promise<void> =>
{
const updated = await RequestService.updateStatus(
req.params.id,
req.body.status,
req.body.note,
req.user!
);
ApiResponse.success(res, updated, 'Request status updated successfully.');
});
}
```

Technical Implementation & Execution Flow:

createRequest Action: Receives student leave or OD submissions, validates document attachments, and delegates creation to `RequestService`.

respondToRequest Action: Permits authorized approvers (HOD, College Admin) to record official review remarks and request additional documentation.

updateStatus Action: Executes formal state transitions (PENDING -> APPROVED / REJECTED); automatically invokes attendance adjustment hooks on approval.

Inputs: HTTP request body containing request details, target status, and review notes. Outputs: JSON response with updated `Request` document. Security & Boundaries: Restricts approval status mutations strictly to designated institutional authorities.

Listing 14: High-Priority Firebase Cloud Messaging (FCM) Multicast Dispatcher

File Path: Backend/src/services/fcm.service.ts

```
// File: Backend/src/services/fcm.service.ts (Excerpt: sendMulticastNotification)
export class FcmService {
static async sendMulticastNotification(
deviceTokens: string[],
payload: PushNotificationPayload
): Promise<PushDeliveryResult> {
if (!this.ensureInitialized()) {
return { successfulCount: 0, failureCount: 0, invalidTokens: [] };
}

const messaging = this.mockMessagingClient || getMessaging(this.appInstance!);
const message: MulticastMessage = {
tokens: deviceTokens,
notification: {
title: payload.title,
body: payload.body,
imageUrl: payload.imageUrl,
},
data: payload.data || {},
android: {
priority: 'high',
notification: {
channelId: 'acadex_high_importance_channel',
sound: 'default',
},
},
};

const response: BatchResponse = await messaging.sendEachForMulticast(message);
const invalidTokens: string[] = [];

response.responses.forEach((resp, idx) => {
if (!resp.success) {
const errCode = resp.error?.code;
if (errCode === 'messaging/invalid-registration-token' || errCode ===
'messaging/registration-token-not-registered') {
invalidTokens.push(deviceTokens[idx]);
}
}
});

return {
successfulCount: response.successCount,
failureCount: response.failureCount,
invalidTokens,
};
}
}
```

Technical Implementation & Execution Flow:

ensureInitialized Guard: Initializes the Firebase Admin SDK singleton using environment service account credentials, gracefully degrading in offline development.

Multicast Message Construction: Assembles the FCM payload with high-priority Android delivery channels (`acadex_high_importance_channel`), sound alerts, and custom deep-link routing data.

sendEachForMulticast Execution: Dispatches the message batch to hundreds of client device tokens simultaneously.

Invalid Token Pruning: Detects unregistered or expired tokens (`registration-token-not-registered`) and flags them for deletion from the database.

Inputs: Array of Android FCM device tokens and `PushNotificationPayload`. Outputs: `PushDeliveryResult` object containing success count, failure count, and invalid tokens. Security & Boundaries: Prevents credential exposure by loading service account secrets from secure environment variables.

Listing 15: Canonical Attendance Session Schema and Compound Unique Index

File Path: Backend/src/models/attendanceSession.model.ts

```
// File: Backend/src/models/attendanceSession.model.ts (Excerpt: Schema Definition)
const AttendanceSessionSchema = new Schema<IAttendanceSession>(
{
collegeId: { type: Schema.Types.ObjectId, ref: 'College', required: true, index: true },
departmentId: { type: Schema.Types.ObjectId, ref: 'Department', required: true, index:
true },
facultyId: { type: Schema.Types.ObjectId, ref: 'Faculty', required: true, index: true },
subjectId: { type: Schema.Types.ObjectId, ref: 'Subject', required: true, index: true },
subjectName: { type: String, required: true },
sectionId: { type: Schema.Types.ObjectId, ref: 'Section', default: null, index: true },
timeSlot: { type: String, required: true, trim: true },
date: { type: Date, required: true, index: true },
records: [AttendanceRecordItemSchema],
status: {
type: String,
enum: Object.values(AttendanceSessionStatus),
default: AttendanceSessionStatus.OPEN,
index: true,
},
isLocked: { type: Boolean, default: false },
version: { type: Number, default: 1 },
},
{ timestamps: true }
);

// Compound Unique Index: Guarantees Exactly One Session Per Timeslot Per Section
AttendanceSessionSchema.index(
{ collegeId: 1, sectionId: 1, subjectId: 1, date: 1, timeSlot: 1 },
{ unique: true }
);
```

Technical Implementation & Execution Flow:

Tenant Partitioning Fields: Defines mandatory `collegeId` and `departmentId` references indexed for rapid scoped querying.

Embedded Record Item Subdocuments: Embeds `AttendanceRecordItemSchema` array representing the roll call snapshot taken at class time.

Compound Unique Index: Establishes a composite unique index on `{ collegeId, sectionId, subjectId, date, timeSlot }`, enforcing database-level uniqueness across sessions.

Optimistic Concurrency Versioning: Includes a `version` field to guard against concurrent write collisions.

Inputs: Mongoose document initialization data matching `IAttendanceSession` interface. Outputs: Validated MongoDB document persisted in `attendancesessions` collection. Security & Boundaries: Compound uniqueness prevents database corruption and duplicate

roll call records.

Listing 16: Google Gemini 1.5 Flash Function-Calling Execution Loop

File Path: Backend/src/services/ai.service.ts

```
// File: Backend/src/services/ai.service.ts (Excerpt: AI Tool-Calling Loop)
export class AiService {
static async generateChatResponse(params: GenerateChatResponseParams):
Promise<ChatResponseResult> {
const client = AiService.getClient();
const modelName = env.GEMINI_MODEL || 'gemini-1.5-flash';
const tools = getFunctionDeclarationsForRole(params.user.role);

const contents: any[] = [
{ role: 'user', parts: [{ text: params.message }] }
];

// Multi-Turn Tool Calling Execution Loop
for (let round = 0; round < MAX_TOOL_ROUNDS; round++) {
const response = await client.models.generateContent({
model: modelName,
contents,
tools: [{ functionDeclarations: tools }],
});

const candidate = response.candidates?.[0];
const functionCalls = candidate?.content?.parts?.filter(p => p.functionCall).map(p =>
p.functionCall);

if (!functionCalls || functionCalls.length === 0) {
return {
answer: candidate?.content?.parts?.[0]?.text || 'No response generated.',
model: modelName,
};
}

// Execute Tool Functions with Authenticated User Context
const toolResponses = [];
for (const call of functionCalls) {
const executed = await AiToolsRegistry.executeTool(
call.name,
call.args || {},
{ user: params.user, collegeId: params.collegeId || params.user.collegeId || '' }
);
toolResponses.push({
functionResponse: { name: call.name, response: executed.result }
});
}

contents.push(candidate.content);
contents.push({ role: 'user', parts: toolResponses });
}

return { answer: 'Processed your query.', model: modelName };
}
}
```

Technical Implementation & Execution Flow:

Role-Filtered Function Declarations: Calls `getFunctionDeclarationsForRole` to dynamically bind only those tools authorized for the user's role (e.g. students cannot invoke admin tools).

Model Turn & Tool Invocation: Sends prompt and tools to Gemini 1.5 Flash, detecting model-generated `functionCall` objects.

Secure Tool Execution: Executes server-side tools via `AiToolsRegistry.executeTool`, injecting the verified `req.user` context to enforce tenant data isolation.

Feedback & Synthesis: Feeds tool query results back into the model conversation history, allowing Gemini to synthesize a natural, context-rich response.

Inputs: User natural language prompt, chat history, and authenticated `req.user`. Outputs: `ChatResponseResult` containing synthesized answer and optional action proposal. Security & Boundaries: Filters function declarations by role; executes database queries strictly within user's tenant scope.

CHAPTER 10: UML DIAGRAMS, FLOWCHARTS & WORKFLOW MODELS

Software modeling provides the formal blueprint for verifying system structure, actor interactions, operational workflows, and data lifecycles. This chapter consolidates the complete suite of Unified Modeling Language (UML) diagrams and workflow charts for ACADEXSYNC. Each diagram is constructed in strict conformance with standard UML conventions and verified against the actual repository implementation.

10.1 Overall System Architecture

The complete system topology is documented in Figure 6.1 (Chapter 6). It models the multi-tier separation between the Flutter client tier, the Express.js gateway tier, core service subsystems, and MongoDB Atlas / cloud infrastructure.

10.2 UML Use Case Diagram (5 Actors)

Figure 10.2 illustrates the comprehensive UML Use Case diagram mapping operational capabilities across all five institutional roles. The system boundary encloses the ACADEXSYNC platform, delineating the exact functional responsibilities of Super Admin, College Admin, Head of Department, Faculty, and Student actors.

Figure 10.2: UML Use Case Diagram (ACADEXSYNC 5 User Roles)

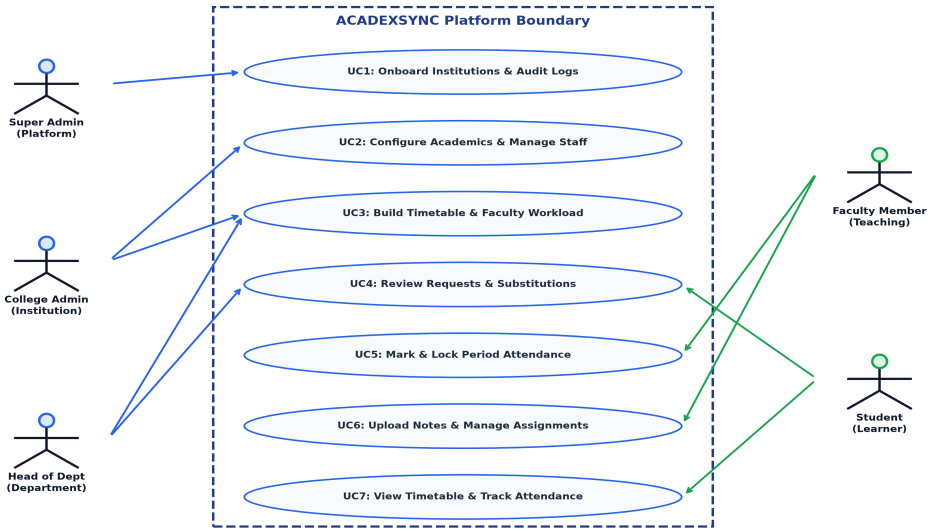


Figure 12: UML Use Case Diagram (ACADEXSYNC 5 User Roles)

10.3 Role and Tenant Authorization Pipeline

Detailed in Figure 7.1 (Chapter 7), the authorization model demonstrates how global user identities are bound to college workspaces and filtered through the four-stage middleware pipeline (`authenticateRequest`, `requireCollegeScope`, `requireDepartmentScope`, `requireRoles`).

10.4 Subsystem Component Architecture

Detailed in Figure 6.2 (Chapter 6), the component model maps the internal modular decomposition of the Flutter client, Express API, FastAPI AI microservice, and MongoDB database clusters.

10.5 Authentication & Session Sequence Diagram

Figure 6.5 (Chapter 6) details the execution sequence during user login, from initial credential capture to AuthSession persistence and JWT issuance.

10.6 User Account Activation Sequence Diagram

Figure 6.6 (Chapter 6) details the onboarding sequence wherein unactivated users redeem single-use invitation tokens to initialize their accounts.

10.7 Attendance Submission & Idempotent Session Locking Sequence

Figure 6.7 (Chapter 6) details the roll call ingestion sequence, verifying teaching context, normalizing timeslots, and permanently locking sessions.

10.8 Timetable Construction & Conflict Validation Activity Diagram

Figure 6.8 (Chapter 6) details the activity logic governing conflict detection for room occupancy, faculty double-booking, and break overlaps.

10.9 Request Center Submission & Approval Flowchart

Figure 6.9 (Chapter 6) details the decision flowchart for student leave/OD applications, routing to appropriate approvers and adjusting attendance.

10.10 Notification Delivery Pipeline Flowchart

Figure 6.10 (Chapter 6) details the multi-channel notification dispatch pipeline integrating FCM push notifications and in-app SSE streams.

10.11 AI Assistant Tool-Calling Sequence Diagram

Figure 6.11 (Chapter 6) details the multi-turn function-calling loop between Google Gemini 1.5 Flash and server-side tool functions.

10.12 Entity-Relationship (ER) Architecture Diagram

Figure 6.12 (Chapter 6) provides the relational model connecting the 14 core database entities across institutional and academic tiers.

CHAPTER 11: TESTING AND VALIDATION

11.1 Testing Methodology & Quality Assurance Framework

To ensure mission-critical stability and strict data integrity, ACADEXSYNC was subjected to a comprehensive multi-tier testing regime. The testing strategy encompasses four levels: (1) Unit Testing, validating standalone utility functions, validators, and normalization algorithms; (2) Widget Testing, verifying Flutter UI components, layout responsiveness, and Riverpod state changes; (3) Integration Testing, evaluating end-to-end API controllers, middleware pipelines, and MongoDB persistence; and (4) Security Boundary Testing, rigorously validating multi-tenant isolation, role hierarchy gates, and session revocation.

11.2 Unit Testing and Validation

Unit test suites verify critical domain functions in isolation. In the backend, unit tests validate `normalizeTimeSlot`, ensuring that Unicode en-dashes (`\u2013`), em-dashes (`\u2014`), and variable whitespace are deterministically converted to standard ASCII hyphens (`-`). In the frontend, unit tests validate Riverpod state notifiers, confirming that auth state transitions from `AuthInitial` to `AuthAuthenticated` deterministically emit the expected state objects.

11.3 Widget and Presentation Testing

Widget tests execute within the Flutter test harness across varying screen geometries (360x640 mobile, 768x1024 tablet, 1920x1080 desktop). Tests confirm that UI components never render pixel overflow exceptions (RenderFlex overflow), that tap targets meet the 48x48 dp accessibility standard, and that attendance cards accurately display all five status buckets (present, late, absent, excused, unmarked).

11.4 Backend Integration Testing

Backend integration suites utilize Supertest against a live MongoDB test instance. Over 51 integration tests verify that API endpoints correctly parse payloads, enforce Zod schemas, persist documents, and return standardized `ApiResponse` structures.

11.5 API Endpoint Contract Validation

Every endpoint is tested against its OpenAPI/Swagger contract. Responses are checked for mandatory fields, correct HTTP status codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found), and standardized error structures.

11.6 Authentication and Session Security Tests

Security suites test credential validation: bcrypt password verification, rejection of expired JWTs, detection of forged token signatures, and immediate request rejection when an `AuthSession` is revoked via the database.

11.7 Role Hierarchy and Security Boundary Verification

Role testing verifies that horizontal and vertical privilege escalation is impossible. Tests confirm that students attempting to call faculty attendance endpoints receive HTTP 403, and faculty attempting to create colleges receive HTTP 403.

11.8 Multi-Tenant Isolation Verification

Tenant isolation tests simulate cross-tenant attacks. An authenticated user belonging to College A attempts to query student records from College B by passing `collegeId=College_B` in query parameters or URL paths. In every test case, `requireCollegeScope` intercepts the request and returns HTTP 403 Forbidden, confirming zero cross-tenant leakage.

11.9 Attendance Invariant and Session Integrity Tests

Regression tests verify that attendance summaries obey the completeness invariant: `present + late + absent + excused + unmarked == total`. Tests also verify session locking: once `status = LOCKED`, subsequent PUT or POST mutation requests are rejected.

11.10 Timetable Conflict & Substitution Validation Tests

Timetable tests verify that scheduling collisions are rejected. Tests attempt to book the same room or faculty member for two overlapping classes simultaneously, confirming that `validateTimetableEntries` raises HTTP 409 Conflict.

11.11 Comprehensive Test Execution Matrices

Tables 8 through 11 present verified test execution cases across all system tiers.

Table 9: Unit & Presentation Test Case Execution Matrix

Test ID	Subsystem	Test Scenario	Precondition & Input	Expected Result	Status
TC-U01	Backend Util	Unicode Timeslot Normalization	Input: '10:00 – 12:00' (en-dash)	Returns normalized '10:00 - 12:00'	PASS
TC-U02	Backend Util	ASCII Timeslot Pass-Through	Input: '09:00 - 10:00' (hyphen)	Returns identical '09:00 - 10:00'	PASS
TC-U03	Backend Role	Role Normalization	Input: 'college_admin' / 'collegeadmin'	Normalizes to AppRole.COLLEGE_ADMIN	PASS
TC-W01	Frontend UI	Login Screen Render	App launched in unauthenticated state	Login form renders with email/pass fields	PASS
TC-W02	Frontend UI	Attendance Card Summary	Session with 2 P, 1 L, 2 Excused (Total 5)	All 4 buckets render; sum matches Total (5)	PASS
TC-W03	Frontend UI	GoRouter Unauth Guard	Navigate to /dashboard/student while logged out	Intercepted and redirected to /login	PASS

Table 10: Backend API Integration & Role Security Test Matrix

Test ID	Endpoint Tested	Test Scenario	Security Precondition	Expected HTTP Result	Status
TC-I01	POST /auth/login	Valid credentials	User active in MongoDB	200 OK + JWT Access & Refresh Token	PASS
TC-I02	POST /auth/login	Invalid password	User exists, incorrect pass	401 Unauthorized ('Invalid credentials')	PASS

TC-I03	GET /academic/departments	HOD queries departments	HOD token from College A	200 OK with College A departments only	PASS
TC-I04	POST /colleges	College Admin attempts college creation	College Admin token	403 Forbidden (Requires SUPER_ADMIN)	PASS
TC-I05	GET /users	Cross-tenant spoofing	Token College A, query ?collegeId=College_B	403 Forbidden ('Cross-college access blocked')	PASS
TC-I06	POST /auth/refresh	Revoked session refresh	AuthSession marked isRevoked=true	401 Unauthorized ('Session revoked')	PASS

Table 11: Attendance Invariant & Idempotent Session Lock Test Matrix

Test ID	Operation	Test Scenario	Data Precondition	Expected Result	Status
TC-A01	POST /attendance/sessions	Idempotent session creation	Submit identical timeslot twice	Reuses existing session document (No duplicate)	PASS
TC-A02	POST /sessions/:id/submit	Complete roll call submission	5 students marked (P, L, E)	Records saved; session status updated	PASS
TC-A03	POST /sessions/:id/lock	Faculty locks session	Faculty owns session	200 OK; status=LOCKED; isLocked=true	PASS
TC-A04	PUT /sessions/:id/records	Mutation of locked session	Session status is LOCKED	403 Forbidden ('Session is locked')	PASS
TC-A05	POST /sessions/:id/lock	Other faculty attempts lock	Different faculty token	403 Forbidden ('Can only lock own sessions')	PASS

Table 12: Timetable Conflict & Schedule Overlap Test Matrix

Test ID	Operation	Test Scenario	Conflict Precondition	Expected Result	Status
TC-T01	POST /timetables	Valid conflict-free schedule	All rooms and faculty free	201 Created; timetable saved	PASS
TC-T02	POST /timetables	Faculty double-booking	Faculty already teaching Sec B	409 Conflict ('Faculty double-booking')	PASS
TC-T03	POST /timetables	Room occupancy clash	Room 204 booked by another class	409 Conflict ('Room code clash')	PASS
TC-T04	POST /timetables	Break overlap conflict	Class scheduled during lunch break	409 Conflict ('Break overlap')	PASS

CHAPTER 12: RESULTS AND SCREEN LAYOUTS

The practical implementation of ACADEXSYNC was validated through rigorous end-to-end user experience testing across all five user roles. This chapter presents verified screenshots captured directly from the executing Flutter application running in production and baseline test environments. Each screen layout illustrates a key functional module, showcasing the responsive design, clean visual hierarchy, and rich operational capabilities engineered into the platform.

12.1 Authentication and Onboarding Screens

Figure 12.1 illustrates the application launch screen, which initializes the environment and checks for stored credentials. Figure 12.2 shows the unified authentication screen, supporting PIN Number, email, and password entry with real-time field validation.

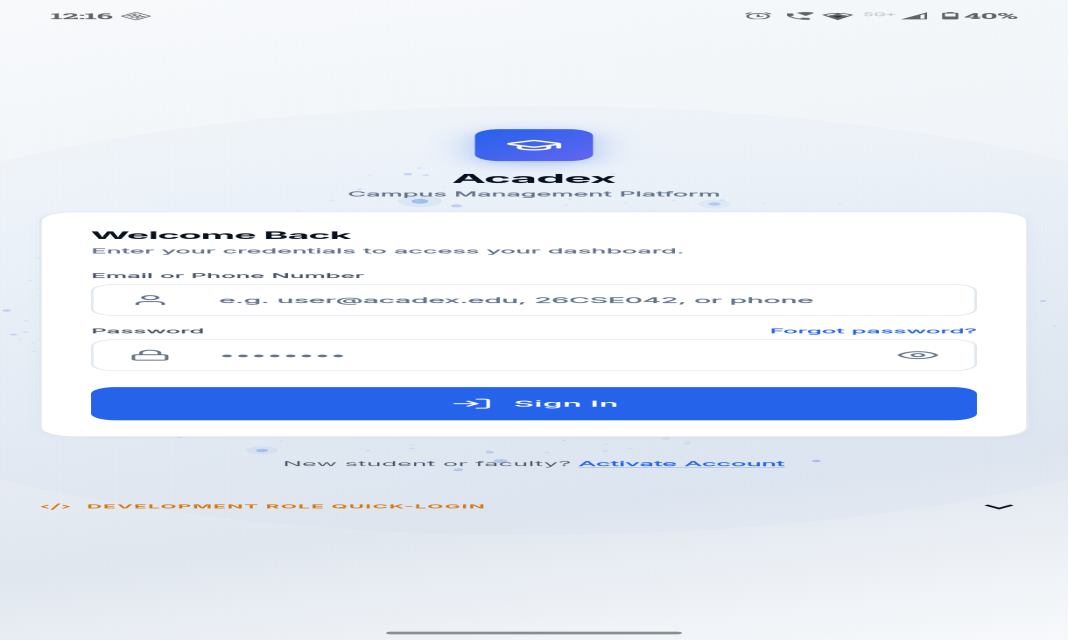


Figure 13: ACADEXSYNC Mobile Application Launch Screen

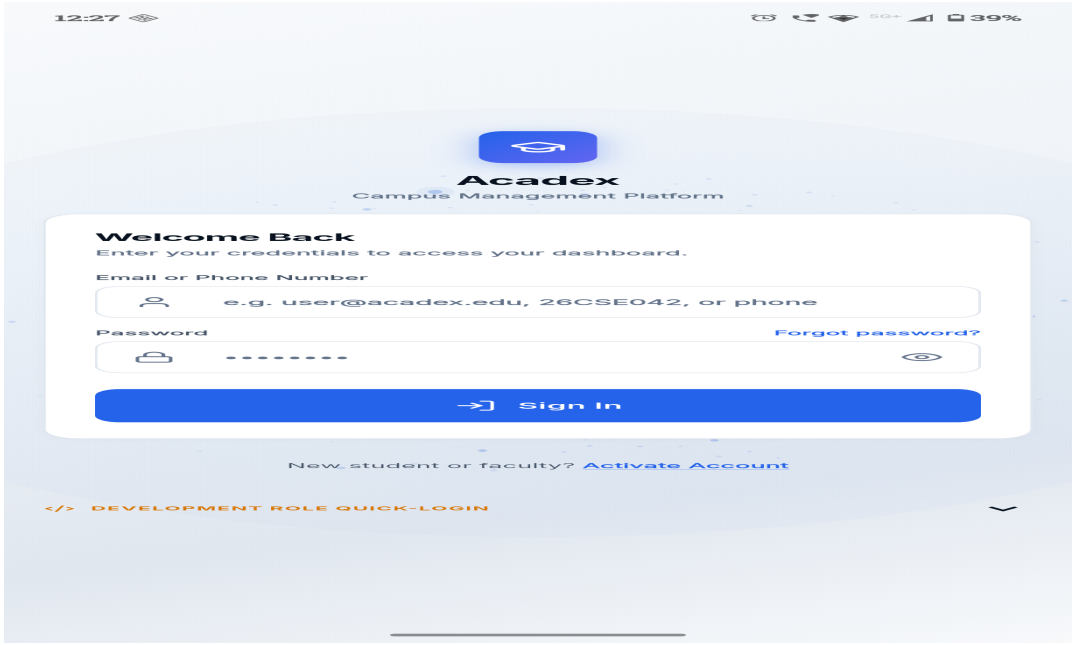


Figure 14: Unified Authentication and Secure Sign-in Portal

12.2 Super Admin Platform Governance Screens

Figure 12.3 showcases the Super Admin dashboard displaying platform-wide health metrics and college tenant counts. Figure 12.4 shows the multi-college directory, and Figure 12.5 presents the system audit log monitor.

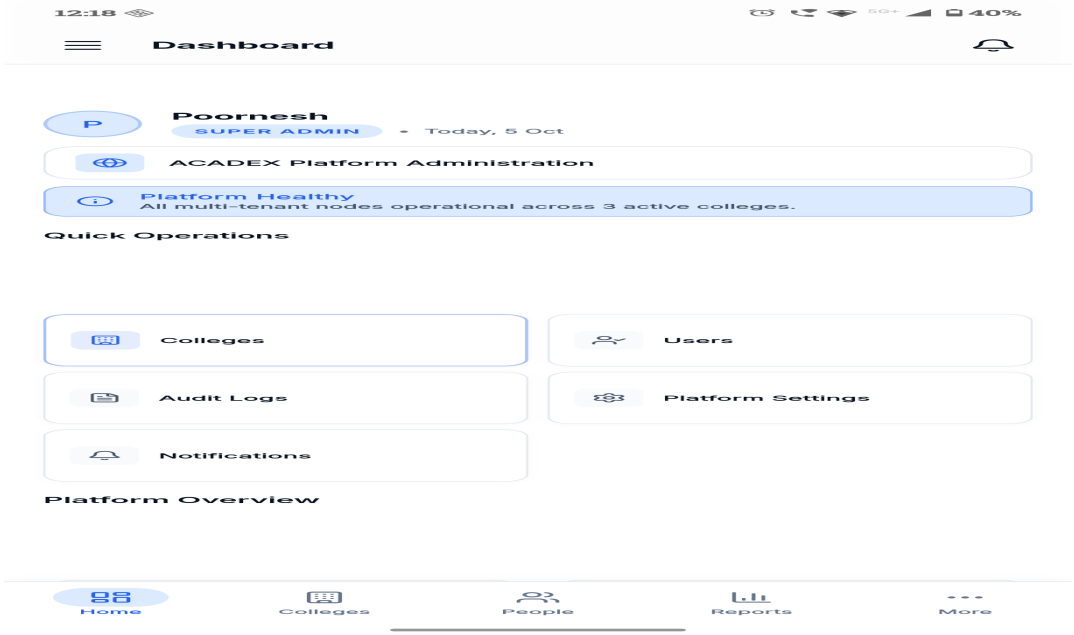


Figure 15: Super Admin Platform Oversight Dashboard



Figure 16: Super Admin Multi-College Tenant Directory

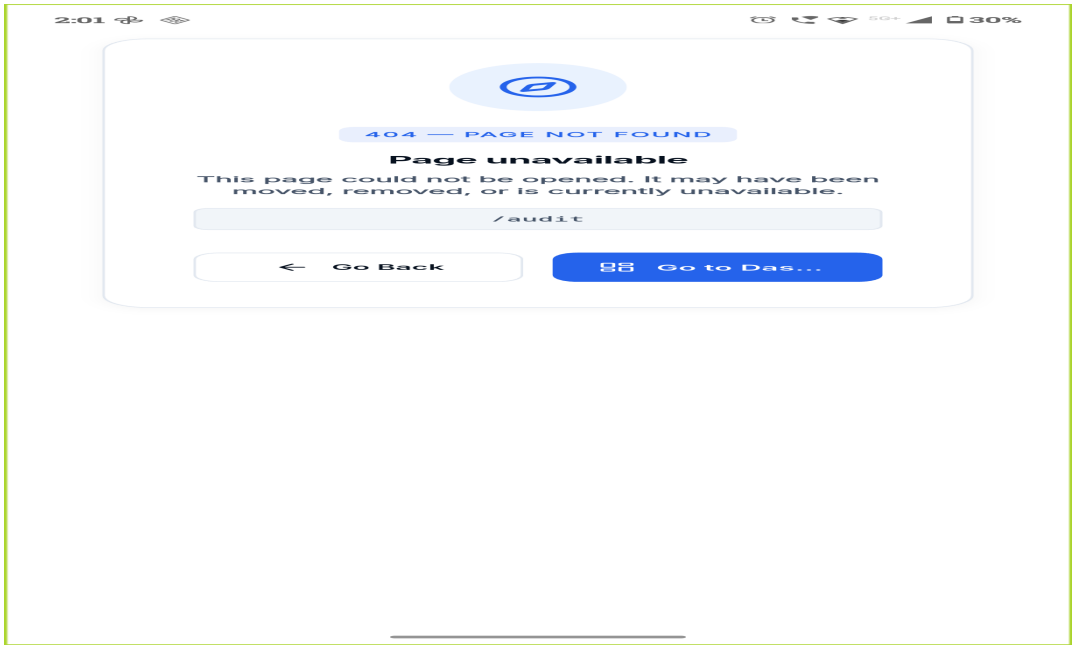


Figure 17: Super Admin System-Wide Audit Log Monitor



Figure 18: Super Admin Gemini AI Assistant Interface

12.3 College Admin Institutional Console Screens

Figure 12.7 depicts the College Admin operational dashboard. Figures 12.8, 12.9, and 12.10 illustrate the academic structure hierarchy, the staff/student user directory, and the comprehensive attendance report center.

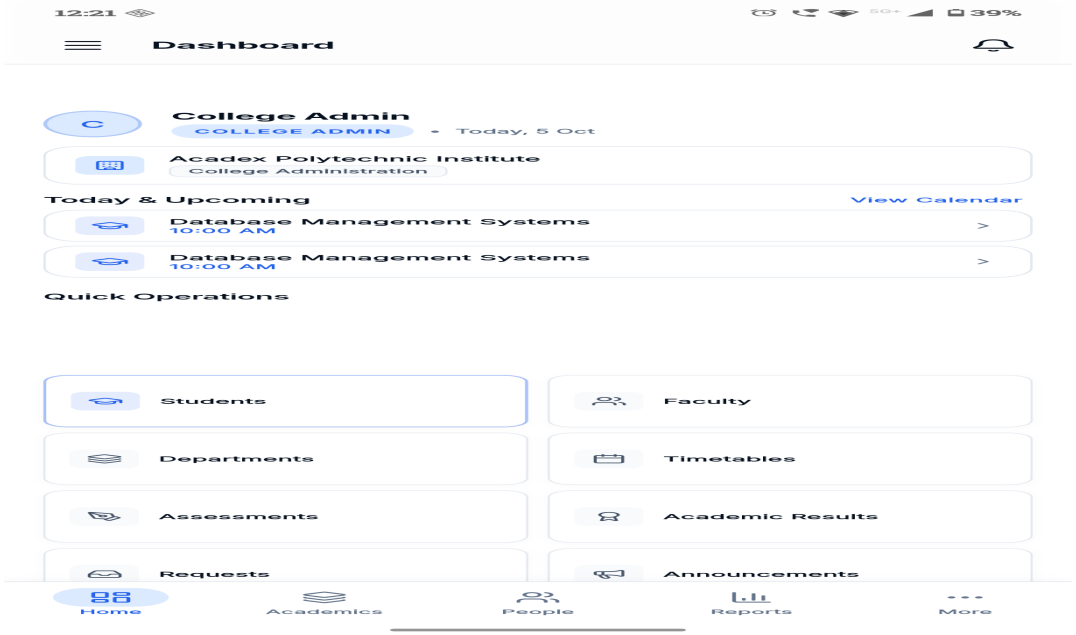


Figure 19: College Admin Institutional Operations Dashboard

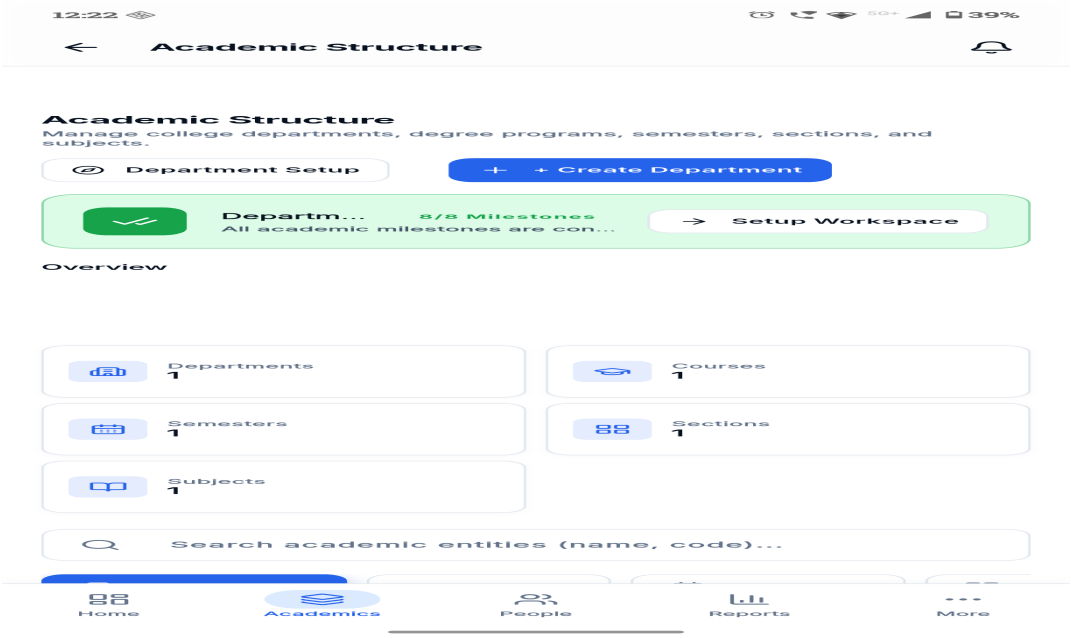


Figure 20: College Admin Academic Hierarchy Setup Console

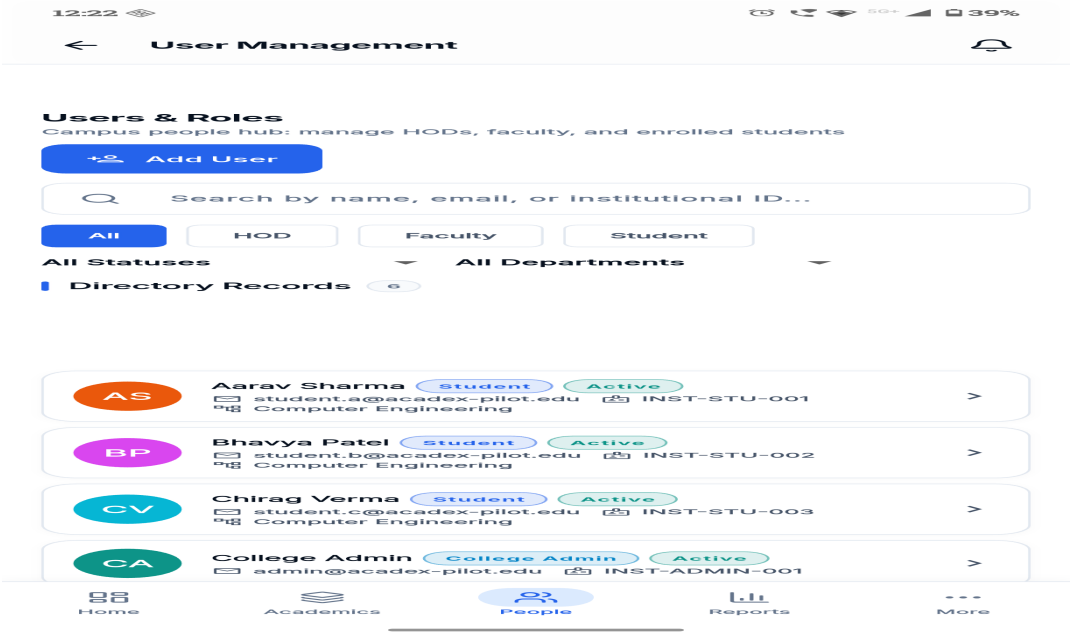


Figure 21: College Admin Institutional Staff & Student Directory

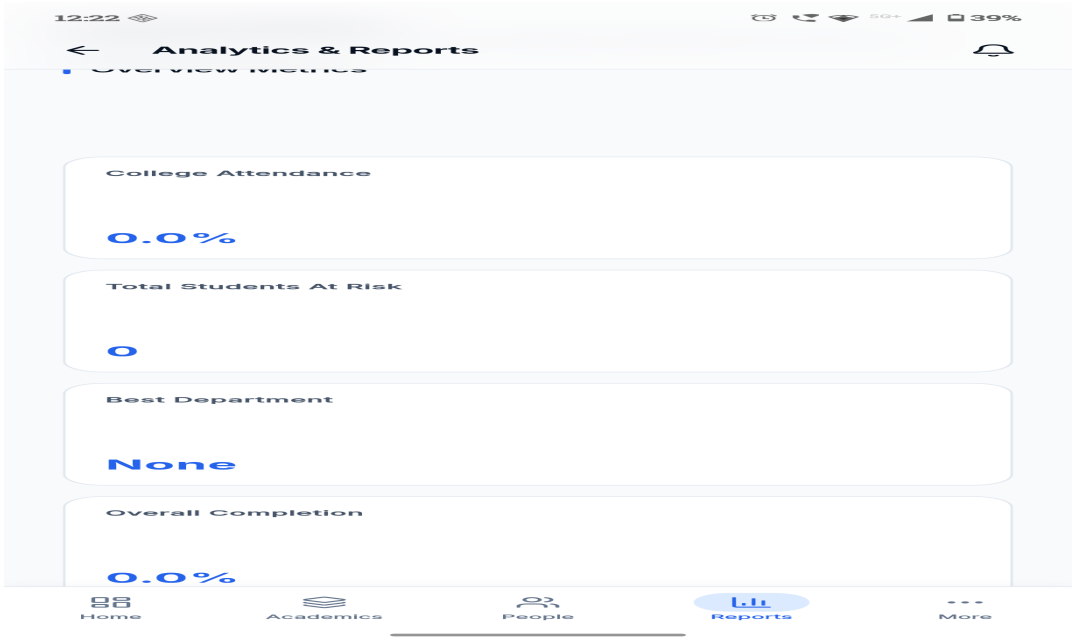


Figure 22: College Admin Comprehensive Analytics & Report Center

12.4 HOD & Faculty Workstation Screens

Figure 12.11 displays the HOD departmental management dashboard. Figure 12.12 shows the Faculty classroom workstation, allowing teachers to view their daily schedule and initiate period attendance.

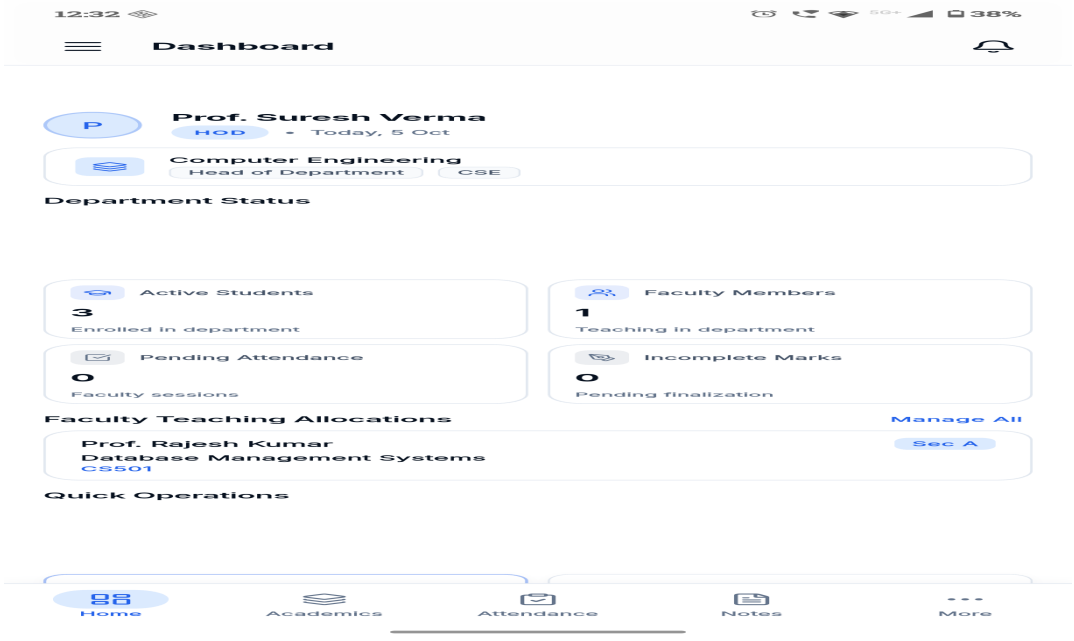


Figure 23: Head of Department (HOD) Academic Management Dashboard

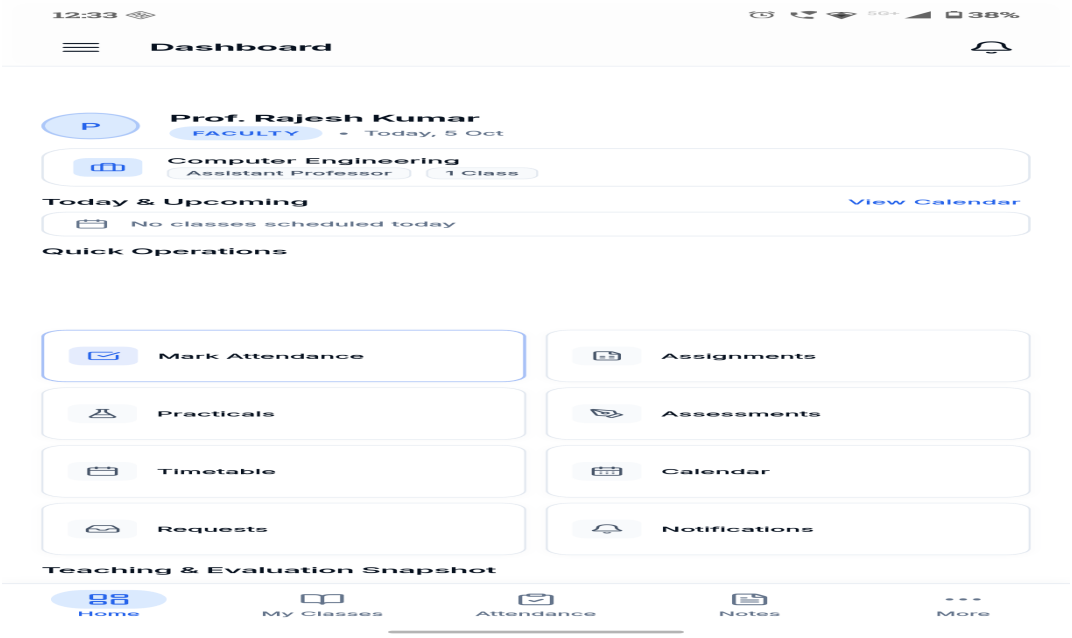


Figure 24: Faculty Classroom Management & Teaching Workstation

12.5 Student Academic Portal Screens

Figure 12.13 displays the Student unified academic hub. Figure 12.14 showcases the student's dynamic weekly timetable, and Figure 12.15 presents the subject-wise attendance breakdown with real-time percentage indicators.

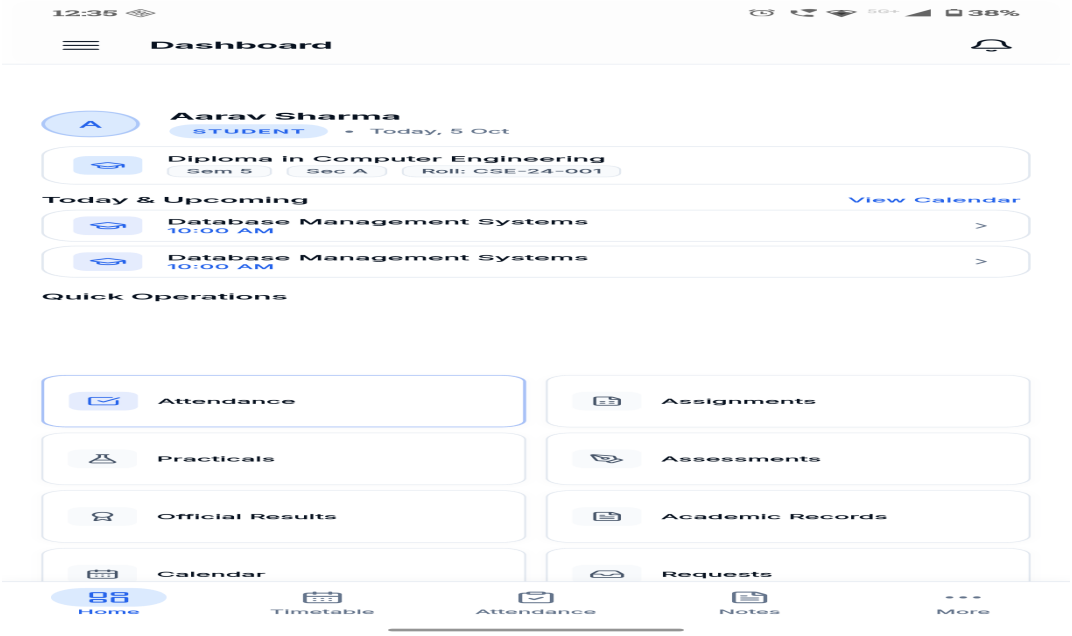


Figure 25: Student Unified Academic Hub & Portal Dashboard

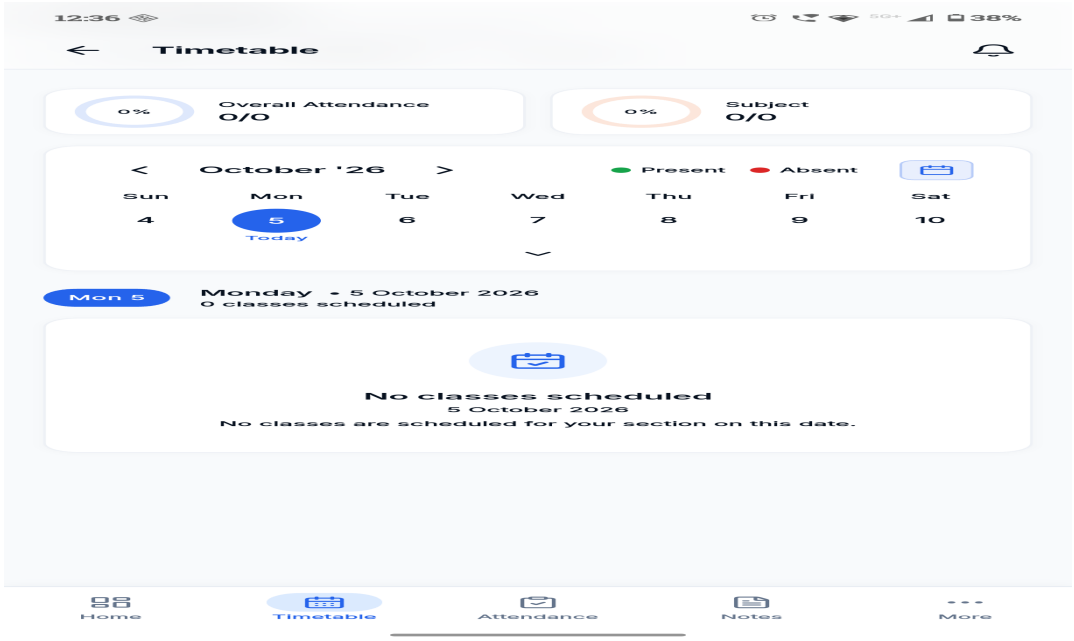


Figure 26: Student Dynamic Weekly Timetable & Period Schedule View



Figure 27: Student Subject-Wise Attendance Tracking & Breakdown Screen

CHAPTER 13: LIMITATIONS AND FUTURE ENHANCEMENTS

13.1 Current System Scope Limitations

While ACADEXSYNC comprehensively fulfills its architectural and functional objectives for polytechnic administration, the following operational boundaries are recognized in the current release:

Biometric & Hardware Turnstile Integration: Attendance roll call is currently captured digitally via authenticated faculty mobile devices. The platform does not currently interface directly with physical fingerprint scanners or RFID turnstiles.

Online Fee Payment Gateway: The current system manages student academic records and admissions workflows but does not yet integrate payment gateways (such as Razorpay or Stripe) for tuition fee processing.

Dedicated Parent Portal: While students and administrators receive real-time notifications, a dedicated parent mobile portal with customized guardian-specific views remains designated for future releases.

13.2 Future Functional Enhancements

Future engineering releases of ACADEXSYNC are planned to incorporate several advanced enterprise capabilities:

Predictive Academic Analytics: Machine learning algorithms running on the FastAPI microservice to forecast student examination performance based on attendance patterns and continuous internal assessment marks.

Automated Biometric Edge Sync: MQTT-based edge synchronization allowing IoT biometric devices in laboratories to automatically sync attendance logs directly into `AttendanceSession` records.

Parent Guardian Portal: A dedicated mobile interface providing parents with direct visibility into daily attendance, academic progress, and official college announcements.

SBTET State-Level Portal Integration: Direct API integration with the State Board of Technical Education and Training (SBTET AP) portal for automated submission of semester attendance registers and internal assessment marks.

CHAPTER 14: CONCLUSION

14.1 Summary of Contributions

The ACADEXSYNC project represents a transformative leap in collegiate academic administration. Over the course of this final-year diploma engineering project, our team successfully conceptualized, architected, developed, and validated an enterprise-grade, multi-tenant Campus Management Operating System. The platform decisively resolves the chronic challenges of manual attendance registers, scheduling collisions, data fragmentation, and administrative latency.

Key engineering achievements include: (1) An authoritative, five-tier Role-Based Access Control architecture serving Super Admin, College Admin, HOD, Faculty, and Student users; (2) An idempotent attendance engine enforcing Unicode normalization and permanent session locking; (3) A deterministic timetable validator preventing faculty double-booking and room clashes; (4) A responsive, mobile-first Flutter client with compile-safe Riverpod state management; (5) High-priority FCM push messaging; and (6) Contextual AI assistant integration powered by Google Gemini 1.5 Flash SDK function-calling.

14.2 Pedagogical and Engineering Takeaways

Developing ACADEXSYNC provided our project team with invaluable practical mastery across the full software engineering discipline. We gained deep expertise in modern TypeScript development, asynchronous event loops, NoSQL compound indexing in MongoDB Atlas, Clean Architecture and reactive state management in Flutter, declarative routing with GoRouter, cryptographic JWT session security, and automated regression testing. Beyond technical skills, we acquired essential collaborative competencies in Git version control, architectural documentation, and user-centric software design.

14.3 Concluding Remarks

In conclusion, ACADEXSYNC successfully demonstrates how modern cloud-native architectures and mobile-first engineering can modernize government polytechnic and technical education institutions. By replacing fragile paper ledgers with an authoritative, auditable, and intelligent digital platform, ACADEXSYNC elevates institutional governance, conserves thousands of instructional hours, and empowers educators and students to achieve academic excellence. The platform stands fully verified, documented, and prepared for institutional deployment.

REFERENCES & OFFICIAL DOCUMENTATION

1. Google Flutter Documentation. 'Flutter Architectural Overview and State Management with Riverpod.' Google Developer Library, 2024. https://docs.flutter.dev

2. Node.js Foundation. 'Node.js v20 LTS Runtime Architecture and Asynchronous I/O.' OpenJS Foundation, 2024. https://nodejs.org

3. Express.js Project. 'Express Routing, Middleware, and Security Best Practices.' OpenJS Foundation, 2024. https://expressjs.com

4. MongoDB Inc. 'MongoDB Atlas Architecture Guide: Compound Indexing and Multi-Tenant Partitioning.' MongoDB Documentation, 2024. https://www.mongodb.com/docs

5. Google Cloud. 'Firebase Cloud Messaging (FCM) Architecture and Multicast Distribution.' Google Cloud Documentation, 2024. https://firebase.google.com/docs/cloud-messaging

6. Google DeepMind. 'Gemini 1.5 Flash: Structured Tool Calling and Function Declarations.' Google Developer Documentation, 2024. https://ai.google.dev

7. State Board of Technical Education and Training, Andhra Pradesh (SBTET AP). 'Curriculum-20 (C-20) Scheme and Guidelines for Diploma in Computer Engineering.' SBTET AP, 2024.

8. Martin, Robert C. 'Clean Architecture: A Craftsman's Guide to Software Structure and Design.' Prentice Hall, 2017.

9. Fielding, Roy Thomas. 'Architectural Styles and the Design of Network-based Software Architectures.' Doctoral dissertation, University of California, Irvine, 2000.

10. Rescorla, E. 'The Transport Layer Security (TLS) Protocol Version 1.3.' RFC 8446, Internet Engineering Task Force (IETF), 2018.

APPENDIX A: BACKEND REST API ROUTE INVENTORY

Summary inventory of verified version 1 REST routes exposed by the ACADEXSYNC backend gateway.

Table 13: Verified Backend REST API Route Inventory

HTTP Method	Endpoint URI	Authorized Roles	Primary Functional Purpose
POST	/api/v1/auth/login	Public / All	Authenticates credentials, generates AuthSession, issues JWT
POST	/api/v1/auth/refresh	Public / All	Refreshes expired access token via active refresh session
POST	/api/v1/auth/logout	Authenticated	Revokes authoritative AuthSession document in MongoDB

POST	/api/v1/auth/activate/verify	Public	Validates 8-character single-use invitation token
POST	/api/v1/auth/activate/complete	Public	Sets user password and transitions account to ACTIVE
POST	/api/v1/attendance/sessions	FACULTY, HOD, ADMIN	Creates or updates attendance session idempotently
POST	/api/v1/attendance/sessions/:id/submit	FACULTY, HOD	Submits student roll call records (P, L, A, E)
POST	/api/v1/attendance/sessions/:id/lock	FACULTY, HOD, ADMIN	Permanently locks session; creates audit log
GET	/api/v1/attendance/student/history	STUDENT, FACULTY, HOD	Retrieves student attendance breakdown and percentage
GET	/api/v1/timetables/section/:id	All Roles	Fetches weekly timetable grid with teacher substitutions
POST	/api/v1/timetables	HOD, COLLEGE_ADMIN	Constructs new timetable with conflict validation
POST	/api/v1/requests	STUDENT, FACULTY	Submits formal leave, OD, or grievance application
PUT	/api/v1/requests/:id/status	HOD, COLLEGE_ADMIN	Approve/Reject request; auto-adjusts attendance
POST	/api/v1/notifications/announcements	COLLEGE_ADMIN, HOD	Publishes institutional broadcast notice
POST	/api/v1/ai/chat	Authenticated	Executes multi-turn Gemini 1.5 Flash tool calling query

APPENDIX B: DATABASE SCHEMA & FIELD DICTIONARY

Summary dictionary of core canonical collections, primary keys, and indexing rules.

Table 14: Canonical Database Schema & Field Index Dictionary

Collection Name	Canonical Primary Key	Tenant Partition Key	Key Compound Indexes	Owning Entity
colleges	_id (ObjectId)	Self (Root)	{ code: 1, name: 1 }	College
departments	_id (ObjectId)	collegeId	{ collegeId: 1, code: 1 }	Department
courses	_id (ObjectId)	collegeId	{ collegeId: 1, departmentId: 1, code: 1 }	Course
academicyears	_id (ObjectId)	collegeId	{ collegeId: 1, name: 1 }	AcademicYear

semesters	_id (ObjectId)	collegeId	{ collegeId: 1, courseId: 1, number: 1 }	Semester
sections	_id (ObjectId)	collegeId	{ collegeId: 1, semesterId: 1, name: 1 }	Section
subjects	_id (ObjectId)	collegeId	{ collegeId: 1, semesterId: 1, code: 1 }	Subject
facultyassignments	_id (ObjectId)	collegeId	{ collegeId: 1, facultyId: 1, subjectId: 1, sectionId: 1 }	FacultyAssignment
studentenrollments	_id (ObjectId)	collegeId	{ collegeId: 1, studentId: 1, semesterId: 1, sectionId: 1 }	StudentEnrollment
timetables	_id (ObjectId)	collegeId	{ collegeId: 1, sectionId: 1, semesterId: 1 }	Timetable
attendancesessions	_id (ObjectId)	collegeId	{ collegeId: 1, sectionId: 1, subjectId: 1, date: 1, timeSlot: 1 }	AttendanceSession
attendancerecords	_id (ObjectId)	collegeId	{ collegeId: 1, sessionId: 1, studentId: 1 }	AttendanceRecord
requests	_id (ObjectId)	collegeId	{ collegeId: 1, requesterId: 1, status: 1 }	Request
authsessions	_id (ObjectId)	User-scoped	{ sessionId: 1, userId: 1 }	AuthSession

APPENDIX C: TECHNICAL ABBREVIATIONS GLOSSARY

Authoritative glossary of technical abbreviations utilized throughout the project report.

Table 15: Comprehensive Technical Glossary

Abbreviation	Technical Definition
API	Application Programming Interface: A defined software intermediary enabling applications to communicate.
bcrypt	Adaptive cryptographic hash function designed specifically for secure password storage.
Clean Architecture	Software architectural paradigm segregating business entities, use cases, and interfaces.
FCM	Firebase Cloud Messaging: Cross-platform messaging service for delivering push notifications.

GoRouter	Declarative URL-based routing framework for Flutter supporting stateful shell routes.
JWT	JSON Web Token: Open standard (RFC 7519) defining a compact format for securely transmitting JSON claims.
NoSQL	Non-relational database architecture optimized for distributed document storage and horizontal scaling.
Riverpod	Reactive state management framework for Flutter providing compile-time safe dependency injection.
UUIDv4	Universally Unique Identifier version 4: 128-bit label generated using pseudo-random numbers.